

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - ВАРНА
Факултет по Изчислителна Техника и Автоматизация
Катедра “Електронна техника и микроелектроника”

ДИПЛОМНА РАБОТА

На тема

FPGA базирана система за диагностика на мрежово оборудване
FPGA-based system for network equipment diagnostics

Дипломант:
/инж. Кристиян Михалев/
Ф. номер: 24651002
Специалност: Електроника

Ръководител:
/Доц. д-р инж. Ф. Ферадов/

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - ВАРНА

ФАКУЛТЕТ ПО ИЗЧИСЛИТЕЛНА ТЕХНИКА И АВТОМАТИЗАЦИЯ

Катедра ЕТМ **Утвърдил:**

Ръководител катедра …… **Декан**

…….……..

/ доц. д-р инж. Т. Папанчев/ / доц. д-р инж. М. Александрова/

З А Д А Н И Е

ЗА ДИПЛОМНА РАБОТА

ОКС „Магистър”

на студента Кристиян Галинов Михалев

фак. № 24651002

1. Тема на проекта: FPGA базирана система за диагностика на мрежово оборудване
   (FPGA-based system for network equipment diagnostics)

2. Изходни данни за проекта:
   - Проектиране на цифров блок за серийна комуникация;
   - Проектиране на цифров блок за интерфейс (клавиатура);
   - Проектиране на цифров блок за интерфейс (дисплей);

3. Съдържание:

   3.1. Литературен обзор на тематиката.
   3.2. Проектиране на блокова схема.
   3.3. Проектиране и изчисляване на системата.
   3.4. Оценка на устройството и резултати.

4. Обем на чертежите: принципна електротехническа схема, спецификация, печатна платка

5. Консултант:

Дата на задаване: 13.10.2025 год.

Срок на предаване: 10.01.2026 годРъководител: .....

Съдържание

Съдържание 3
Увод 6
Глава 1 - Литературен обзор на тематиката 9
1.1 Методи за достъпване на диагностични данни 9
1.2 Решения на проблема, налични на пазара 10
1.2.1 Решения, базирани на микроконтролери 10
1.2.2 Ретро-компютърни и образователни решения 12
1.2.3 FPGA-базирани решения 13
1.3 Технология за реализация 14
1.4 Развойна платка 16
1.5 Визуализиране на данните 17
1.6 Интерфейс за вход на данни 20
1.7 Сериен интерфейс 23
Глава 2 - Проектиране на блокова схема 27
2.1 Архитектурна философия 27
2.2 Подсистема за обработка на входове 29
2.3 Подсистема за управление на данните 30
2.4 Подсистема за визуализация 32
2.5 Организация на данните и йерархия на паметта 33
2.5.1 Текстов буфер 34
2.5.2 Знаков генератор 34
2.5.3 Графичен видео буфер 35
2.6 Хардуерна платформа и физическа среда 36
2.6.1. Избор на хардуерна платформа 36
2.6.2 Периферни интерфейси и физически ограничения 39
2.6.3 Електрически стандарти и съгласуване на нивата 40
2.6.4 Конфигурация и енергонезависима памет 41
2.6.5 Оценка на използваните ресурси 41
2.7 Обобщение на архитектурните решения 43
Глава 3 – Проектиране и изчисляване на системата 45
3.1 Цифрова система – RTL реализация 45
3.1.1 Обща структура на RTL дизайна 47
3.1.2 PS/2 интерфейс (ps2_keyboard_to_ascii.vhd) 52
3.1.3 UART интерфейс 54
3.1.4 Централен модул за управление на данните (data_handler.vhd) 57
3.1.5 Генератор на кадри (frame_builder.vhd) 64
3.1.6 LCD драйвер (lcd_driver_de_nv3047_with_bram.vhd) 70
3.1.7 Паметни блокове (BRAM) и IP конфигурация 74
3.1.8 Обобщение на RTL реализацията 78
3.2 Хардуерна реализация – дъщерна платка 79
3.2.1 Концепция за интеграция и механично изпълнение 80
3.2.2 Функционални блокове на дъщерната платка 81

3.2.3 Интерфейс към TFT LCD дисплея ... 81
3.2.4 Захранване на LED подсветката на дисплея ... 82
3.2.5 Сериен интерфейс RS-232 ... 82
3.2.6 PS/2 интерфейс и резервиран USB порт ... 83
3.2.7 Реализация на печатната платка ... 84
3.2.8 Обобщение и препратка към приложенията ... 84
3.3 Оценка на използваните хардуерни ресурси ... 85
3.3.1 Логически ресурси (LUT и регистри) ... 85
3.3.2 Използване на LUT като памет (LUTRAM) ... 86
3.3.3 Използване на блокова памет (BRAM) ... 86
3.3.4 DSP и специализирани ресурси ... 87
3.3.5 Тактови и входно-изходни ресурси ... 87
3.3.6 Особености при отчитането на IP ядра ... 87
3.3.7 Обобщение и инженерни изводи ... 87
Глава 4 - Оценка на устройството и резултати ... 89
4.1 Тестова постановка ... 89
4.1.1 Физическа конфигурация ... 90
4.1.2 Измервателни средства ... 91
4.2 Верификация на модула data_handler.vhd ... 91
4.2.1 Цел на измерването ... 91
4.2.2 Методика на измерването ... 92
4.2.3 Наблюдавани сигнали ... 93
4.2.4 Резултати от измерването ... 93
4.2.5 Анализ и интерпретация ... 94
4.3 Физическо измерване и верификация на UART предаване ... 95
4.3.1 Цел на измерването ... 95
4.3.2 Методика на измерването ... 95
4.3.3 Наблюдавани сигнали ... 96
4.3.4 Резултати от измерването ... 97
4.3.5 Анализ и интерпретация ... 98
4.4 Физическо измерване и верификация на модула frame_builder.vhd ... 98
4.4.1 Цел на измерването ... 98
4.4.2 Методика на измерването ... 99
4.4.3 Конфигурация на тригера и наблюдавани сигнали ... 100
4.4.4 Резултати от измерването и анализ ... 101
4.4.5 Изводи от измерването ... 102
4.5 Физическо измерване и верификация на LCD драйвера ... 103
4.5.1 Вътрешно наблюдение чрез ILA ... 103
4.5.2 Външно физическо измерване чрез осцилоскоп ... 104

4.5.3 Съпоставка и интерпретация 105
4.5.4 Бележка относно HSYNC и VSYNC сигналите 105
4.5.5 Извод от измерването 106
4.6 Обобщение и изводи от физическите измервания 106
Заключение 108
Използвана литература 110
Спецификация 112
Приложения 114

Увод

От 90-те години насам, с глобалното разпространение на интернет, компютърните мрежи поетапно биват внедрени в ежедневието на хората. Тези мрежи непрекъснато се разрастват както по брой, така и по сложност, пропорционално на увеличаващия се брой потребители и устройства, свързани към глобалната мрежа. Нарастващата нужда от свързване на милиони потребители, както и широката дигитализация на работни места, производствени предприятия, търговски обекти и разнообразни услуги, изисква непрекъснато разширяване, модернизация и професионална поддръжка на мрежовата инфраструктура. Постепенно все повече компании преминават към предлагане на услуги в интернет — електронна търговия, софтуерни платформи, облачни решения за съхранение и обработка на данни и много други.

През последните години, системите с изкуствен интелект разширяват възможностите си с големи темпове, което води до необходимостта от големи изчислителни мощности и бързо нарастване на броя и капацитета на центровете за данни. Въпреки че голям процент от световното население ежедневно използва интернет и стотици онлайн услуги, малък процент от тях осъзнават сложността и мащаба на хардуера, който осъществява тяхната работа, както и ролята на инженерите, поддържащи тези съоръжения и осигуряващи тяхната денонощна работа. Това повишава значението на експертите по компютърни мрежи - професионалисти, които трябва да осигурят надеждна работа, бърза реакция при проблеми и непрекъснато оптимизиране на инфраструктурата.

С разрастването на мрежите расте и нуждата от висококачествени диагностични инструменти, позволяващи бърз анализ, локализиране на проблеми и мониторинг на устройства на място, в реално време. Въпреки това, голяма част от съществуващите решения разчитат на операционни системи, универсален софтуер или специализиран хардуер с висока цена и ограничена гъвкавост. Това създава затруднения при поддръжката на критични мрежови системи и увеличава времето за реакция при неизправности. Липсва достатъчно компактно, надеждно и лесно преносимо оборудване, предназначено специално за нуждите на мрежовите инженери.

В този контекст програмируемите логически матрици (FPGA) се явяват обещаваща платформа за създаване на ново поколение диагностични устройства. Те позволяват паралелно изпълнение на голям брой хардуерни процеси, предоставят бързодействие, висока предсказуемост на времето за реакция и възможност за доразвитие на функционалността без промяна на електронния хардуер. Благодарение на това FPGA решенията са особено подходящи за анализ на данни от мрежово оборудване, визуализация на параметри в реално време и взаимодействие с различни комуникационни интерфейси.

Настоящата дипломна работа разглежда проектирането и реализацията на прототипна FPGA-базирана система за диагностика на мрежово оборудване. В тази връзка, основните цели на настоящата дипломна работа са разработването на прототипна FPGA базирана система за диагностика на мрежово оборудване, способна да приема, визуализира, изпраща диагностични данни от и към мрежово оборудване в реално време, без разчитане на външни компютърни ресурси и без използване на операционна система.

Системата трябва да бъде компактна, надеждна, подходяща за мрежови инженери, да осъществява поддръжка, анализ, тестване на мрежова инфраструктура.

За постигане на поставената цел се определят следните задачи:

- Детайлен анализ на съществуващите решения в областта.
- Проектиране на хардуерната архитектура и развойна система – определяне на логическите блокове в системата.
- Разработка на цифрова система (HDL) и развойна платформа (печатна платка).
- Хардуерна реализация на модулите – синтез и имплементация
- Тестване в реална среда с мрежово оборудване.

Глава 1 - Литературен обзор на тематиката

1.1 Методи за достъпване на диагностични данни

Мрежовото оборудване, през целия експлоатационен период, трябва да може да бъде управлявано чрез някакъв интерфейс – било то за настройка на параметрите и опциите, наблюдаване на производителността на уреда, или предоставяне на диагностични данни при настъпване на повреда или абнормално поведение. Надеждността и достъпността на този интерфейс са критични за поддръжката на инфраструктурата.

Този интерфейс може да бъде осигурен по няколко начина, класифицирани според нивото на достъп и зависимостта им от мрежовата свързаност:

- **Уеб-базиран интерфейс**: Достъпван през локалната мрежа чрез IP адреса на устройството (HTTP/HTTPS).
- **Мрежови протоколи**: Стандартизирани протоколи за отдалечен достъп като SSH, Telnet или SNMP.
- **Заводска услуга**: Достъпвана през специализиран софтуер (Vendor-specific), например MikroTik WinBox, Ubiquiti UniFi Network Controller или Cisco Packet Tracer.
- **Физически конзолен порт**: Директна хардуерна връзка чрез интерфейси като RJ45 console, RS232 DB9, USB to UART или Mini-USB.

Макар мрежовите методи (SSH, Web) да са удобни за ежедневна администрация, те имат ограничение: зависими са от работоспособността на самата мрежа. В ситуации на критични аварии (boot loops, грешна конфигурация на firewall/VLAN,

корупция на фърмуера), мрежовият стек на устройството може да бъде недостъпен.

В тези сценарии физическият конзолен порт остава възможен метод за диагностика (Out-of-band management). Традиционното решение за достъп до този порт – използване на лаптоп с USB-преходник – е свързано с редица неудобства: големи габарити, необходимост от зареждане на операционна система и зависимост от драйвери. Това обуславя практическата приложимост на специализирано, компактно и автономно устройство (ръчен терминал), което да осигурява незабавен достъп до конзолния порт.

1.2 Решения на проблема, налични на пазара

В сферата на диагностичната апаратура за мрежови устройства липсват специализирани, бюджетни хардуерни терминали, които да се предлагат като готов търговски продукт. Стандартната практика в индустрията е да се разчита почти изцяло на използването на преносим компютър с емулатор на терминал (PuTTY, TeraTerm) и преходник USB-RS232. Този подход, макар и универсален, е свързан с високо енергопотребление, големи габарити и зависимост от операционна система.

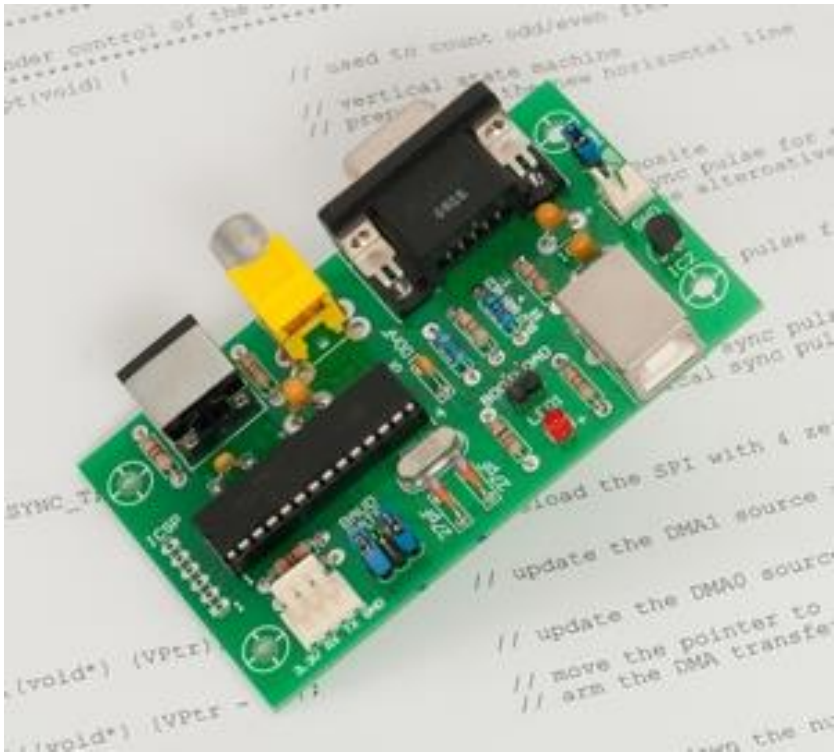
В общността на отворения хардуер (Open Source Hardware) и любителската електроника съществуват опити за решаване на този проблем. Те могат да бъдат категоризирани в три основни групи, всяка от които има своите предимства и технологични ограничения.

1.2.1 Решения, базирани на микроконтролери

Най-популярният подход за създаване на евтини терминали е използването на 32-битови микроконтролери.

- **Geoff's VT100 Terminal:** Този проект реализира емулация на стандарта DEC VT100 върху PIC32 (или по-късно ARM)

микроконтролер. Системата приема вход от PS/2 клавиатура и генерира композитен или VGA видео сигнал. Основното ограничение е последователната обработка на данните. Микроконтролерът трябва да разпределя процесорното си време между приемането на UART данни, обработката на сигналите от клавиатурата и генерирането на видео сигнала. При висок трафик (напр. наплив от логове при стартиране на рутер) и висока скорост (115200 baud), софтуерният цикъл често не успява да обработи прекъсванията навреме, което води до загуба на символи или артефакти в изображението.



Фигура 1 - Geoff's VT100 Terminal

- **VT2040 (RP2040 based):** Проект, използващ чипа Raspberry Pi Pico. Въпреки наличието на две ядра, архитектурата остава софтуерно-зависима и податлива на латентност при натоварване, характерна за всички MCU решения.



Фигура 2 - VT2040 (RP2040 based)

1.2.2 Ретро-компютърни и образователни решения

Съществуват проекти, целящи възраждането на стари архитектури с образователна цел.

- **6802 Serial Terminal:** Базиран на реални микропроцесори от 70-те години (Motorola 6802) и LED матрични дисплеи. Тези проекти имат висока историческа стойност, но са неприложими за съвременна мрежова диагностика поради изключително ниската резолюция на дисплея и големите габарити на използваните DIP корпуси.



Фигура 3 - 6802 Serial Terminal

1.2.3 FPGA-базирани решения

Решенията с програмируема логика са най-близки до целите на настоящата дипломна работа, тъй като предлагат хардуерна стабилност.

- **FPGA Serial Terminal (Howard Jones / fpga-vt):** Проектът реализира VT100 терминал върху FPGA чипове (Cyclone II/Spartan-3). Този проект е реализиран чрез VHDL е сходен като принцип на реализация. Основният недостатък на проекта е, че той е разработен с цел извеждане на сигнал към стандартен VGA монитор. Тази негова характеристика го обуславя като стационарно устройство, неподходящо за работа в сървърни шкафове или на труднодостъпни места.

Направеният анализ показва наличие на пазарна ниша за устройства, които комбинират надеждност, компактност и преносимост. В литературата и практиката се срещат разработки, насочени към автономни преносими терминали с вграден дисплей и

работа без операционна система, като подход за улесняване на диагностика на мрежово оборудване.

1.3 Технология за реализация

За реализацията на такова устройство е необходимо да се направи обоснован избор на изчислителна архитектура и периферия. Основното архитектурно решение при проектирането на вградена система е изборът между микропроцесорна (MCU) и хардуерна (FPGA) реализация.

За сравнение на технологиите е необходимо да се разгледат специфичните изисквания на видео-терминала:

1. **Високоскоростен вход:** Приемане на данни по UART (често 115200 baud или повече) без загуба на пакети.
2. **Визуализация:** Опресняване на цветен LCD дисплей (минимум 60Hz) без трептене.
3. **Латентност:** Мигновена реакция при натискане на клавиш.

В Таблица 1 е представено сравнение на двете технологии спрямо тези критерии.

Таблица 1 - сравнение между микроконтролер и програмируема логика

Характеристика	Микроконтролер (MCU)	Програмируема логика (FPGA)
Обработка на данни	Последователна: CPU изпълнява инструкции една след друга.	Паралелна: Всички модули работят едновременно и независимо.
Видео сигнал	Изисква DMA и натоварва системната шина. Риск от "накъсване" при натоварване.	Генерирането на синхросигнали е хардуерно детерминирано. Идеална стабилност.
Реакция (Latency)	Зависи от прекъсвания (Interrupt Latency).	Нулева (в рамките на 1 такт).
Надеждност	Възможност за софтуерни грешки (Stack overflow, exception).	Хардуерна стабилност.

Микроконтролерите, макар и по-евтини, споделят изчислителния ресурс между комуникацията и видеото. При

интензивен трафик от рутера (напр. boot log), съществува риск MCU да пропусне байтове докато обновява дисплея. FPGA технологията позволява създаването на независими хардуерни блокове – един за UART прием и друг за видео контролер – които работят паралелно без конкуренция за ресурси. Това е предпоставка за по-предсказуемо поведение при натоварване.

При разглеждане на конкретни реализации следващата стъпка е селекцията на конкретен чип. Критичният параметър за видео приложения без външна памет е обемът на вградената памет (Block RAM / BRAM), необходима за съхранение на видео буфера (Framebuffer) и знакогенератора.

Разглеждат се три популярни фамилии в бюджетния и средния клас:

- Xilinx Spartan-7 / Intel Cyclone 10 LP**:** Тези чипове са оптимизирани за ниска цена, но разполагат с ограничен обем BRAM (обикновено под 1-2 Mbits). Това може да наложи използването на външна SDRAM памет за видео буфер, което усложнява дизайна значително (например необходимост от DDR контролер).
- Xilinx Artix-7: Тази фамилия предлага значително по-голям ресурс. Конкретно моделът XC7A100T разполага с 4,860 Kbits BRAM.

  В Таблица 2 е представено сравнение на кандидатите, с фокус върху ресурсите, необходими за видео обработка.

Таблица 2 - сравнителна характеристика на FPGA чипове

Характеристика	Xilinx Artix-7 (XC7A100T)	Xilinx Spartan-7 (XC7S50)	Intel Cyclone 10 LP (10CL025)
Логически клетки	101,440	52,160	25,000
Вградена памет (BRAM)	4,860 Kb	2,700 Kb	594 Kb
Достъпност на платки	Висока (QMTech, Digilent)	Средна	Ниска
Пригодност за видео	Отлична (Голям BRAM за framebuffer)	Добра	Ограничена (Изисква външна RAM)

За съхранение на текстова матрица, атрибути за цвят и знакогенератори, както и за потенциално разширение към графичен режим, е необходим значителен обем бърза памет. По-големият обем BRAM позволява част от тези функции да се реализират с вътрешни ресурси и да се намали зависимостта от външна памет и съпътстващите контролери.

1.4 Развойна платка

След дефиниране на необходимия изчислителен ресурс, физическата реализация на прототип на вградена система често се базира на готова развойна платформа (development kit) или модулен „core board“. Това е разпространена практика, тъй като платформата интегрира критични за висикоскоростните цифрови системи блокове – захранване с множество напрежения, тактова система и високочестотни трасета за външна памет – които при собствена платка изискват значителни усилия по проектиране, опроводяване и верификация.

Типичните изисквания към развойна платка за терминално устройство с дисплей включват наличие на FPGA с достатъчен логически ресурс и вградена памет, както и изведени входно-изходни линии за периферия. Често се търси и външна оперативна

памет (например DDR), която позволява реализация на по-големи видео буфери, графични режими или съхранение на обемни диагностични данни. За съхранение на конфигурацията и постоянни ресурси се използва енергонезависима памет (например SPI Flash), която може да съдържа конфигурационния битстрийм и таблици със символи/шрифтове.

От гледна точка на интерфейсите, развойните платки често предоставят видео изход (HDMI/VGA или паралелен RGB), както и стандартни конектори/рейки, които извеждат GPIO пиновете и позволяват свързване на специализирана периферия (клавиатурен интерфейс, UART/RS-232, нива за преобразуване и др.). Наличието на модулна архитектура и стандартни конектори улеснява изграждането на „носител“ (carrier) или адаптерна платка, върху която се добавят специфичните за приложението интерфейси.

Използването на готова платформа намалява вероятността от хардуерни грешки, свързани със захранването на FPGA ядрата и високочестотните връзки, и позволява усилията да се насочат към реализацията и верификацията на функционалните блокове на системата и необходимите диагностични интерфейси.

1.5 Визуализиране на данните

Визуализацията на данните е ключов компонент в системата на преносимия мрежов терминал. За разлика от настолните компютри, където графичният адаптер и мониторът са отделни устройства, при вградените системи дисплеят е неразделна част от архитектурата. Неговите параметри диктуват изискванията към паметта, тактовата честота и логическите ресурси на FPGA чипа.

За да изпълнява ефективно функциите си на диагностичен инструмент за конзолен достъп, дисплейната система трябва да отговаря на следните критерии:

- **Разделителна способност и четимост:** Стандартният конзолен екран изисква визуализация на минимум 60–80 символа на ред, за да бъдат съобщенията четими без накъсване. При използване на стандартен шрифт 8×16 пиксела, това налага минимална хоризонтална резолюция от 480 пиксела (8 пиксела × 60 символа = 480 пиксела). Вертикалната резолюция се изчислява по аналогичен начин – чрез определен брой редове, които трябва да бъдат визуализирани. 15 реда са достатъчни, за да се вижда най-скорошната история от съобщения, следователно вертикалната резолюция е практично да бъде поне 240 пиксела (16 пиксела шрифт × 15 реда = 240 пиксела).
- **Скорост на опресняване (Refresh Rate):** Текстовата информация от мрежовите устройства често се генерира с висока скорост. Дисплеят трябва да поддържа стабилно опресняване от 60 Hz, за да се избегне ефектът на "мазане" (ghosting) или накъсване (tearing).
- **Независимост от външни контролери:** Тъй като системата се реализира върху FPGA без специализиран графичен процесор (GPU), управлението на дисплея трябва да става чрез директно генериране на синхронизиращи и даннови сигнали от програмируемата логика.
- **Компактност:** Устройството е преносимо, което ограничава физическия размер, но дисплеят трябва да бъде достатъчно голям, за да бъде лесно четимо съдържанието и да не е изморително продължително четене. Типичен диапазон е между 4 и 6 инча.

На база на дефинираните изисквания, бе направен анализ на наличните технологии за свързване на LCD матрици към FPGA. В Таблица 3 са разгледани трите основни стандарта.

Таблица 3 - сравнение на технологии за свързване на LCD матрици

Характеристика	**Сериен (SPI / I2C)**	**Високоскоростен сериен (HDMI / MIPI DSI)**	**Паралелен RGB (DPI / Parallel Interface)**
Принцип на работа	Изпраща пикселите последователно по 1-2 линии.	Диференциални двойки с висока честота и кодиране.	Паралелно предаване на всички цветови битове + CLK/SYNC.
Скорост (Bandwidth)	Ниска. Трудно постига 60Hz при резолюции над 320x240.	Изключително висока (Gbps).	Висока (съвпада с Pixel Clock).
Сложност на реализация	Ниска (малко пинове).	Много висока (Изисква SERDES блокове, лицензирани IP ядра).	Умерена (Изисква само броячи и синхронизация).
Натоварване на FPGA	Високо (изисква сложни машини на състоянията за инициализация).	Високо (консумация на логика за протокола).	Минимално (Директно управление на I/O пиновете).
Пригодност за проекта	Неподходящ (бавно скролиране).	Неподходящ (ненужна сложност).	Оптимален.

- **SPI дисплеите са ограничени** от пропускателната способност. За резолюция 480×240 при 60 Hz е необходим поток от данни над 120 Mbps, което е на границата на SPI контролерите и създава риск от латентност.
- **HDMI/MIPI решенията** предлагат висока пропускателна способност, но **обикновено** изискват по-сложна реализация и специализирани хардуерни блокове, които не са налични във всички FPGA платформи.
- **Паралелният RGB интерфейс** представлява балансиран подход. Той позволява на FPGA чипа да генерира изображението в реално време, пиксел по пиксел, синхронно с тактовия генератор, като използва директно управлявани I/O линии.

При избор на дисплеен модул обичайно се задават минимални параметри за резолюция и интерфейс, след което се подбират конкретни панели, които ги покриват. Пример за често срещана конфигурация е TFT LCD панел в диапазона 4–6 инча с резолюция около 480×272 пиксела и 24-bit паралелен RGB интерфейс (RGB888). Интерфейсът използва 8 пина за всеки цветови канал (червен, зелен, син) плюс управляващи сигнали (HSYNC, VSYNC, DOTCLK, DE), което позволява 16.7 милиона цвята (True Color).

Дисплеите от този клас обикновено се управляват от драйверна интегрална схема, която приема стандартизирани синхронизиращи и даннови сигнали и позволява автономна работа на системата след инициализация.

1.6 Интерфейс за вход на данни

За да бъде една диагностична система пълноценна, тя трябва да осигурява възможност за бързо и удобно въвеждане на текстови команди. Мрежовото оборудване (Cisco, MikroTik, Juniper) се

управлява предимно чрез команден интерфейс (CLI), който изисква пълна QWERTY клавиатура с поддръжка на специални символи и клавишни комбинации.

При проектирането на вградени системи изборът на подходящ интерфейс за клавиатура е ключово решение. То трябва да балансира между удобството за потребителя и сложността на хардуерната реализация.

За сравнение на приложимите подходи при реализация върху програмируема логика (FPGA) може да се разгледа сравнителен анализ на трите основни стандарта за свързване на клавиатури: USB (HID), матрични клавиатури и PS/2. В Таблица 4 са съпоставени характеристиките на всеки от тях спрямо критериите за автономност и ефективност на ресурсите.

Таблица 4 - сравнителни характеристики на входни устройства

Характеристика	**USB Клавиатура (HID)**	**Матрична клавиатура**	**PS/2 Клавиатура**
Метод на комуникация	Polling(Опитване): Хостът трябва постоянно да запитва устройството,което ангажира изчислителен ресурс.	Scanning(Сканиране): Изисква постоянен цикъл за проверка на редове и колони от страна на контролера.	Interrupt(Прекъсване): Устройството е пасивно и изпраща сигнал само при събитие (натискане).
Физически слой	Диференциална двойка (D+/D-) с висока честота. Изисква външен PHY чип или сложен аналогов интерфейс.	Множество цифрови пинове (GPIO). Например 8 пина за едва 16 бутона.	2 пина (Clock и Data) + Захранване. Изключително икономичен откъм пинове.
Сложност на имплементация (FPGA)	Висока: Изисква реализация на USB Host Stack, Enumeration процес и обработка на дескриптори. Трудно за изпълнение без процесор.	Ниска: Просто управление на I/O, но изисква логика за премахване на трептенето (Debouncing) и е ограничена като брой клавиши.	Умерена: Синхронен сериен протокол. Лесен за декодиране чрез краен автомат (FSM) и таблица (LUT).
Приложение	Потребителска периферия, PC.	Калкулатори, охранителни панели, домофони.	Вградени системи, Лаптопи, Индустриални терминали.

Матричните клавиатури са ограничени при сценарии, в които е необходима пълна азбучна подредба (QWERTY) за писане на команди. USB интерфейсът, макар и масово разпространен, води до по-висока сложност при система без операционна система, поради необходимостта от реализиране на хост функционалност и протоколен стек. PS/2 стандартът се отличава с ниска хардуерна сложност и минимален брой линии, като същевременно осигурява функционалността на стандартна клавиатура.

Често срещано заблуждение е, че PS/2 протоколът е технологично остарял, тъй като външният конектор вече не присъства в масовите лаптопи и настолни компютри. В действителност обаче, този стандарт запазва приложимост във вътрешната архитектура на мобилните компютри.

В съвременните лаптопи връзката между вградената клавиатура и дънната платка рядко се осъществява чрез USB. Вместо това се използва специализиран вграден контролер (Embedded Controller – EC), който комуникира с клавиатурната матрица и предава данните към централния процесор чрез протокол, съвместим с класическия PS/2 (чрез емулация на i8042 контролер).

Технологичните причини за запазването на този стандарт са валидни и днес:

- Архитектура, базирана на прекъсвания (interrupt-driven): При PS/2 устройството само инициира комуникация при натискане на клавиш. Това поддържа ниска латентност, за разлика от USB, където хостът периодично запитва устройството.
- Независимост от драйвери ("instant-on"): PS/2 клавиатурите функционират на ниско хардуерно ниво веднага след подаване

на захранване. Това е важно за достъп до BIOS/UEFI настройки или boot менюта, преди да е заредена операционна система.

В контекста на проектирането на независим хардуерен терминал, използването на PS/2-съвместим входен интерфейс може да се разглежда като възпроизвеждане на утвърден подход от архитектурата на преносимите компютри. Това подпомага надеждността и бързодействието и оставя възможност за интегриране на компактна клавиатура от лаптоп към FPGA платформа.

1.7 Сериен интерфейс

Ефективната диагностика на мрежово оборудване изисква задълбочено познаване на физическите интерфейси за управление. В следващите параграфи се разглеждат стандартите за серийна комуникация, примерна тестова постановка с оборудване от серията MikroTik RouterBOARD и принципите за проектиране на съгласуваща електроника между него и FPGA система.

Въпреки навлизането на USB интерфейсите, серийната комуникация остава широко използван стандарт за локално управление (Local Management) на корпоративен мрежов хардуер. Тя осигурява достъп до конзолата (CLI) на ниско ниво, независимо от състоянието на операционната система на устройството.

В индустрията са се наложили няколко основни реализации на физическо ниво:

- **EIA/TIA-232 (RS-232) с конектор DE-9 (DB9):** Класическият стандарт, използващ 9-пинов D-Sub конектор. Среща се при по-старо оборудване и индустриални рутери (напр. MikroTik RB400/RB800 серия, стари Cisco устройства). Използва пълни

напреженови нива (±12V) и често поддържа хардуерен контрол на потока (RTS/CTS).

- **TIA-561 (RJ45 Console / "Cisco Style"):** Използва стандартен 8-пинов модулен конектор (RJ45) и плосък "Rollover" кабел. Де факто стандарт за Cisco, Juniper, Ubiquiti и модерните устройства на MikroTik (напр. CCR серията). По-компактен от DB9, но електрическите сигнали остават съвместими с RS-232 стандарта.
- **USB Serial (USB-to-UART Bridge):** Устройствата имат вграден преобразувател (напр. FTDI или CP210x) и USB конектор (Mini-B, Micro-B, Type-C). Открива се при модерно домашно и SOHO оборудване. Изисква наличие на драйвери от страна на терминала (Host), което усложнява реализацията върху FPGA без процесор.

При реализация на терминално устройство без операционна система често се разглежда RS-232 с DB9 конектор, тъй като предоставя директен достъп до Tx/Rx линиите без необходимост от USB стек и обработка на дескриптори.

Анализ на комуникационния интерфейс на MikroTik RouterBOARD

Като примерна платформа за тестване може да се разгледа рутер от серията MikroTik RouterBOARD 4xx (напр. RB433). Тези устройства са широко разпространени в лабораторни и обучителни среди и притежават класически сериен порт.

Серията RB4xx е оборудвана с мъжки конектор DB9 (DTE – Data Terminal Equipment). Това е важно уточнение, тъй като определя типа на необходимия кабел. Когато се свързва компютър (също DTE) към рутер (DTE), е необходим Null Modem (Crossover) кабел, който кръстосва предавателната (TX) и приемателната (RX) линия. При терминално устройство, което функционира като DTE,

връзката към DTE устройство също следва да бъде кръстосана (cross-over).

Според техническата документация на MikroTik, параметрите за комуникация по подразбиране са:

- Скорост (Baud Rate): 115200 bps
- Битове за данни: 8
- Паритет: Няма
- Стоп битове: 1
- Flow Control: Off (изключен по подразбиране за конзолата, въпреки че хардуерът поддържа RTS/CTS).

Тази конфигурация (115200/8N1) дефинира изискванията към сериината комуникация за проекта.

Основното предизвикателство при свързването на FPGA към RS-232 порт е несъвместимостта на електрическите нива. Стандартът RS-232 дефинира логическите нива чрез биполярни напрежения спрямо земя (GND):

- Логическа "0" (Space): +3V до +15V (обикновено +12V при MikroTik).
- Логическа "1" (Mark): -3V до -15V (обикновено -12V).

Входно-изходните блокове (IOB) на FPGA чипа работят с нисковолтова логика (LVTTL / LVCMOS):

- Логическа "0": 0V
- Логическа "1": 3.3V

Директното опроводяване на сигналните линии от DB9 конектора към FPGA може да доведе до пробив на входните транзистори и трайна повреда на чипа. Необходимо е междинно звено – преобразувател на нива (Level Shifter). За решаването на този проблем обикновено се разглеждат три подхода:

- **Дискретна реализация с транзистори:** Използване на схема с биполярни транзистори (BJT) или MOSFET за инвертиране и ограничаване на напрежението. Изисква външно биполярно захранване (±12V) за генериране на изходния сигнал към рутера, което усложнява захранващия блок на платката. Ниска шумоустойчивост.
- **Използване на MAX232:** Класическият чип MAX232 е индустриален стандарт от десетилетия. Предимството му е, че има вградена електроника (charge pump), която генерира двуполярно захранване от еднополярно захранване. При 3.3V логика може да възникне необходимост от допълнително преобразуване на нива между чипа и FPGA, което увеличава сложността.
- **Използване на MAX3232:** Чипът MAX3232 е еволюция на MAX232, оптимизирана за нисковолтови системи. Работи в широк диапазон (3.0V до 5.5V). При захранване с 3.3V логическите му входове/изходи са директно съвместими с FPGA. Използва зарядна помпа и обикновено изисква няколко външни кондензатора (напр. 0.1 μF), за да генерира необходимите RS-232 напрежения.

В практиката комуникационният блок често включва RS-232 трансивър (например MAX3232-клас) и подходящ конектор (DB9), като кръстосването на линиите може да бъде реализирано чрез кабел (null modem) или чрез опроводяване на платката, в зависимост от ролите DTE/DCE. При подобна реализация е типично да се предвиди захранване от основната система и да се осигури запас по скорост спрямо 115200 bps, който покрива стандартните конзолни режими на мрежово оборудване.

Глава 2 - Проектиране на блокова схема

Проектирането на специализирана вградена система върху програмируема логика (FPGA) изисква фундаментално различен подход спрямо класическото софтуерно инженерство. Докато софтуерните системи разчитат на последователното изпълнение на инструкции от централен процесор, хардуерната архитектура се базира на пространствен паралелизъм и синхронизация на множества от процеси.

Настоящата глава дефинира архитектурната рамка на системата, като разглежда декомпозицията на функционалните блокове, потоците от данни и стратегията за управление на ресурсите. Проектирането се извършва на две абстрактни нива:

1. **Логическо ниво (FPGA Design):** Описание на вътрешната структура, тактовите домейни, машините на състоянията (FSM) и алгоритмите за обработка на данни.
2. **Физическо ниво (Hardware Design):** Проектиране на електрическите вериги на развойната система и печатната платка (PCB).

2.1 Архитектурна философия

За разлика от стандартната компютърна архитектура тип „фон Нойман“, при която входът, обработката и видеоизходът се конкурират за ресурсите на един централен процесор и обща шина, настоящият проект възприема принципите на поточната архитектура (Dataflow Architecture).

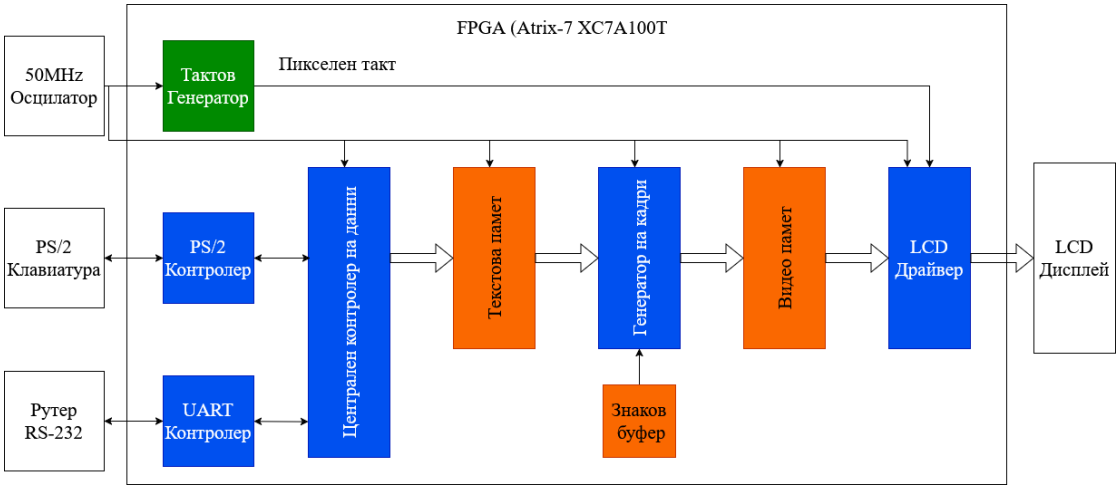
Този подход е пряко свързан с изискванията на терминално устройство, при което едновременно протичат три процеса: прием на символен поток по сериен интерфейс (типично 115200 bps или

повече), непрекъснато опресняване на дисплей (60 Hz) и незабавна реакция при клавиатурен вход. При последователна микропроцесорна обработка тези задачи споделят един и същ изчислителен ресурс и се управляват чрез прекъсвания и/или DMA, което при високо натоварване може да доведе до пропуснати байтове или до нарушена плавност на визуализацията. При FPGA реализацията функционалността се разделя на независими хардуерни блокове, работещи паралелно, което подпомага детерминираното поведение на системата.

Основните концептуални принципи, залегнали в проектирането, са:

- **Разделяне на домейните (Domain Separation):** Процесите по генериране на видео сигнал са напълно отделени от процесите по обработка на потребителски команди. Това гарантира, че дори при тежко натоварване на системата (напр. бързо писане или получаване на много данни от рутера), изображението на екрана няма да трепти или прекъсва.
- **Специализирани хардуерни ускорители:** Вместо софтуерни алгоритми, се използват специализирани логически блокове за конкретни задачи – модул за управление на паметта, модул за графично рендиране и модул за комуникация.
- **Липса на операционна система:** Системата работи на ниво "Bare-metal hardware", което елиминира латентността (закъснението) и осигурява мигновена реакция при стартиране ("Instant-on").

Въз основа на функционалните изисквания, цифровата система в FPGA чипа е разделена на три основни подсистеми, работещи в синхрон.



Фигура 4 – Структура на дизайна за реализация в FPGA чипа

2.2 Подсистема за обработка на входове

Тази подсистема функционира като входен интерфейс, който изолира вътрешната логика от сложността на физическите протоколи. За разлика от стандартните решения, където централният процесор се занимава с декодиране на протоколи, тук архитектурата разчита на предварителна обработка (Pre-processing).

- **Интерфейс за данни (Data Interface):** Входът към тази подсистема е проектиран на ниво "символен поток" – при него модулът получава готов 7-битов ASCII код (диапазон 0–127 по опростената ASCII таблица) и управляващ сигнал "Valid Strobe" (прекъсване за нов символ). Това архитектурно решение позволява пълна независимост от типа на клавиатурата – системата би работила идентично и с USB клавиатура, стига да има преобразувател до същия ASCII интерфейс. В използваната реализация входът от клавиатура се осигурява чрез PS/2 интерфейс, като отделен модул декодира PS/2 сканкодове и ги преобразува до ASCII на изхода на този блок. По този начин останалата част от системата работи единствено със символи и

управляващи събития, без да е обвързана с протоколните детайли на клавиатурата.

- **Логика на събитията (Event Logic):** Тъй като входните данни са вече декодирани до знаци, ролята на този блок е семантична класификация. Той анализира постъпващия ASCII код и определя типа на събитието:
    1. Текстово събитие (Data Event): Стандартни буквено-цифрови символи, които се насочват към буфера за визуализация.
    2. Управляващо събитие (Control Event): Специални кодове (напр. 0x0D за Enter, 0x08 за Backspace), които не се визуализират директно, а задействат процедури в логиката на курсора или паметта.
- **Комуникационен буфер (UART Interface):** Аналогично на клавиатурата, комуникационният вход също се третира като поток от ASCII символи. Подсистемата осигурява арбитраж, позволявайки смесване на входа от потребителя (локална клавиатура) и входа от мрежовото оборудване (рутер) в един общ информационен поток към екрана.

2.3 Подсистема за управление на данните

Тази подсистема е централното изчислително звено в архитектурата. Тя не просто съхранява данни, а активно управлява пространствената организация на информацията върху екрана. Функционалността ѝ е декомпозирана на три взаимосвързани модула:

- **Логика на курсора (Cursor Logic)**: Този модул поддържа текущото състояние на потребителската позиция (координати X и Y). Той е проектиран като двумерен брояч с гранични условия,

който следи дали позицията на писане не излиза извън видимата област на екрана. Логиката е пряко свързана със следващия модул – когато курсорът достигне края на последната линия, той генерира сигнал за препълване, който служи като тригер за скролиране. Координатите на курсора се интерпретират в символни единици (колона/ред) и са съобразени с текстовата организация на екрана (например 60 колони по 17 реда), което позволява адресиране на текстовия буфер на ниво символ.

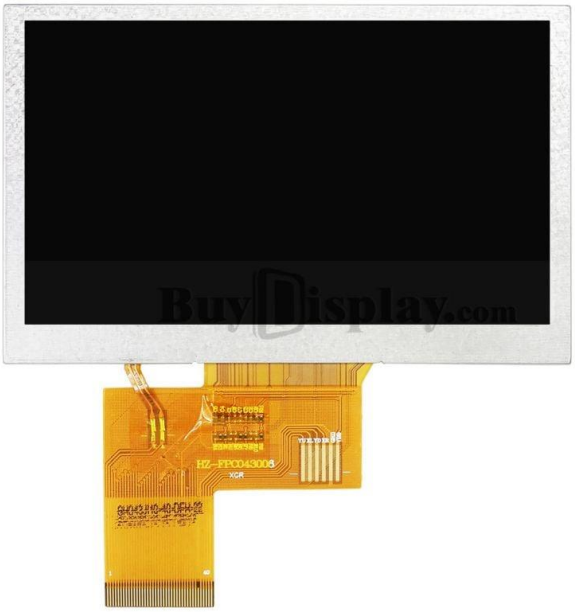
- **Механизъм за скролиране (Scrolling Logic)**: Това е специализиран хардуер, проектиран да реши проблема с преместването на масиви от данни. Вместо софтуерен цикъл, който би копирал байт по байт, механизмът използва паралелен достъп до паметта. Той се активира автоматично от логиката на курсора и изпълнява операция "Read-Modify-Write" върху цели редове, изместващи съдържанието на екрана вертикално нагоре с минимална латентност. Операциите по преместване се изпълняват върху цели редове от текстовия буфер, което е съгласувано с неговата ширина и позволява ефективно пренасяне на данните при скролиране.
- **Арбитър на паметта (Memory Arbiter)**: Тъй като паметта е споделен ресурс, този модул предотвратява конфликти (Race Conditions). Той дава приоритет на операциите по запис от страна на клавиатурата или UART порта, гарантирайки, че потребителският вход никога не се губи, дори по време на опресняване на екрана. Арбитражът е съобразен с това, че визуализационната подсистема извършва паралелно четене на съдържание, докато входните подсистеми и логиката на скролиране извършват записи, което изисква ясно дефинирани приоритети и времево разделяне на достъпите.

2.4 Подсистема за визуализация

Видео трактът е реализиран като еднопосочен конвейер (Pipeline), който работи синхронно с тактовата честота на дисплея. Неговата цел е да осигури непрекъснат поток от пиксели без "накъсване" (tearing) на изображението.

- **Генератор на кадри (Frame Builder):** Този блок извършва транслация в реално време. Той чете ASCII кодовете от текстовата памет и прави справка в знаков генератор (Font ROM), за да получи графичната матрица на съответния символ. Архитектурното решение тук избягва съхранението на пълна битова карта (Bitmap) на целия екран, което драстично намалява изискванията към паметта на FPGA чипа.
- **Видео контролер (LCD Driver):** Този модул е отговорен за физическото управление на TFT панела. Той генерира сложна поредица от синхронизиращи импулси и активни периоди (Data Enable), които дефинират геометрията на изображението (480x272). В използваната конфигурация дисплейният модул е TFT LCD панел с диагонал 4.3 инча и резолюция 480×272 (WQVGA), свързан чрез 24-bit паралелен RGB интерфейс (RGB888). Видео шината включва 8 линии за всеки цветови канал (R, G, B), както и управляващи сигнали DOTCLK (PCLK), DE, HSYNC и VSYNC. Панелът работи в режим DE-only, при който валидната област се дефинира чрез сигнал Data Enable (DE) и пикселен такт DOTCLK (PCLK), а синхронизиращите сигнали HSYNC/VSYNC не се използват за адресиране на активната област (поддържат се в фиксирани нива). Модулът работи с тактовата честота от 9MHz, за да осигури стабилно опресняване при 60 Hz. Дисплейният панел се управлява от интегрална схема драйвер от клас NV3047 (или еквивалент),

която приема стандартните синхронизиращи и даннови сигнали от FPGA.



Фигура 5 - 4.3inch TFT LCD

Описаната визуализационна подсистема осигурява непрекъснат и детерминиран видео поток, но нейното функциониране е пряко зависимо от начина, по който данните са организирани и съхранявани в паметта. За да се гарантира висока пропускателна способност и отсъствие на конфликти между процесите на генериране и визуализация на изображението, е необходимо ясно дефинирано йерархично разделение на данните. Поради тази причина в следващия раздел се разглежда организацията на данните и паметната архитектура на системата.

2.5 Организация на данните и йерархия на паметта

За да се осигури висока производителност и гладка визуализация, архитектурата на системата използва тристепенна йерархия на паметта (Three-stage Memory Hierarchy). Този каскаден подход позволява разделяне на процесите на генериране и

визуализация на изображението, като всеки етап използва специализиран буфер, оптимизиран за конкретната задача.

Потокът на данните преминава през три логически трансформации:

1. **Семантично ниво (Text Buffer):** Съхранение на символни кодове.
2. **Графично ниво (Font ROM):** Преобразуване на кодове в битови матрици.
3. **Пикселно ниво (Frame Buffer):** Буфериране на пикселни данни за физическия дисплей.

2.5.1 Текстов буфер

Този буфер е основното хранилище на информацията в терминала. Той съдържа "карта" на екрана, където всеки адрес от паметта съответства на конкретен ред от дисплея. Буферът съхранява резултата от работата на централния контролер. В него се записват ASCII кодовете, постъпили от клавиатурата или серийния порт.

Паметта е конфигурирана с ширина 480 бита и дълбочина 17 адреса. Тъй като един ред на екрана съдържа 60 символа, а всеки символ е 8 бита, целият ред заема точно 480 бита. Това позволява четене/запис на цял ред за един тактов цикъл. 17-те адреса съответстват на 17-те текстови реда, които могат да се поберат във вертикалната резолюция от 272 пиксела.

2.5.2 Знаков генератор

Това е памет само за четене, която изпълнява ролята на транслатор (Look-Up Table). Тя съдържа графичните дефиниции на всички поддържани символи. Използваният шрифт е с фиксиран размер 8x16 пиксела. Стандартната ASCII таблица съдържа 128

символа. Следователно, необходимият обем е: 128 символа×16 линии/символ=2048 адреса. За да се извлече пикселната репрезентация на даден символ, е необходимо да се генерира 11 битов адрес със следната структура: Старши 7 бита – ASCII кодът на символа, прочетен от текстовия буфер, младши 4 бита – индексът на текущия пикселен ред (0-15) който се изчертава в момента на прочитане. Паметта връща 8 битова дума представляваща стойности на пикселите (включен/изключен) на даден символ за конкретен ред.

2.5.3 Графичен видео буфер

Последното ниво в йерархията на паметта е видео буферът, разположен между генератора на кадри и драйвера на дисплея. За разлика от линейните буфери, които пазят само един ред, тази памет е оразмерена да съхранява пълното графично съдържание на целия екран. Капацитетът на буфера е изчислен така, че да покрие точно физическата резолюция на дисплея (480x272 пиксела), като се използва пакетиране на данните за максимална ефективност. Пикселите на екрана са 480 * 272 = 130 560. Идеята на тази памет е да пази последователно всички стойности на пикселите които трябва да бъдат визуализирани. Тези стойности са комплектовани като стандартна дума от 32 бита, която ще служи за ширина на паметта. Дълбочината на паметта се получава след като разделим броят пиксели на широчината – 130 560/32 = 4080 адреса, като в резултат паметта е изпълнена изцяло с полезна информация и няма празни и неусвоени ресурси. Следователно тази памет е конфигурирана с ширина 32 бита и дълбочина 4080 адреса. Този буфер осигурява пълно развързване и разделяне на процесите на генериране и визуализация. Генераторът на кадри попълва буфера,

като преобразува символите от ASCII кодове в пикселни битове (чрез шрифтовата памет) и ги записва на пакети по 32 пиксела. Видео контролерът чете последователно адресите от 0 до 4079. Прочетените 32-битови думи се сериализират (преобразуват се в поток бит-по-бит) и се подават към дисплея в синхрон с пикселния такт. Това архитектурно решение гарантира изключително стабилно изображение, тъй като дисплеят винаги чете от готов, статичен буфер, елиминирайки риска от графични артефакти при високо натоварване на системата. В текущата реализация графичният буфер е монохромен (1 bpp). При формиране на изхода към RGB888, активният пиксел се разширява до 0xFF за всеки цветови канал (R=G=B=0xFF), а неактивният пиксел се задава 0x00.

Представената организация на данните и йерархия на паметта дефинира логическата структура на системата и изискванията към ресурсите, необходими за нейната реализация. Тези изисквания следва да бъдат съпоставени с физическите възможности и ограничения на използваната хардуерна платформа. В този контекст следващият раздел разглежда избраната развойна система и физическата среда, в рамките на които архитектурата е реализирана.

2.6 Хардуерна платформа и физическа среда

2.6.1. Избор на хардуерна платформа

Архитектурата на разработената система е съобразена с ресурсните възможности и ограничения на избраната развойна платформа. Като физическа основа на проекта е използвана развойната платка QMTech Wukong Board V3, базирана на FPGA чип от фамилията Xilinx Artix-7.

Платформата е изградена около FPGA чип Xilinx Artix-7 XC7A100T-2FGG484I и включва интегрирани памети и интерфейси, характерни за развойни системи от този клас: DDR3 памет MT41K128M16 (256MB, 16-битова шина) и SPI Flash памет с капацитет 32MB, предназначена за съхранение на конфигурационния битстрийм и допълнителни постоянни данни.

Изборът на платформата е продиктуван от необходимостта да се осигури достатъчен логически и паметен ресурс, съчетан с висока степен на интеграция и наличност на стандартни интерфейси. В използваната конфигурация платката предоставя стабилна хардуерна основа за реализацията на сериен терминал, без да въвежда архитектурни ограничения върху логическата структура на системата.

Основният използван програмируем логически елемент е FPGA чипът XC7A100T, който осигурява значителен запас от логически ресурси и вградена памет. Това позволява всички функционални блокове на системата да бъдат реализирани изцяло в рамките на FPGA матрицата, без необходимост от външни контролери или допълнителни програмируеми устройства.

От гледна точка на настоящата разработка, следните ресурси на платформата са от съществено значение:

- **Блокова памет (Block RAM):** наличният обем от вградена памет позволява реализиране на текстови и видео буфери директно в FPGA, което гарантира детерминистично време за достъп и стабилна работа на визуализационната подсистема.
- **Управление на тактовите сигнали**: вграденият 50 MHz генератор, в комбинация със специализираните тактови модули (MMCM), позволява прецизно генериране на всички необходими тактови честоти, включително пикселния такт за LCD дисплея. В

използваната конфигурация пикселният такт за дисплея е 9 MHz, като се получава чрез тактова обработка от референтния източник.

- **Логически ресурси**: наличният брой логически клетки осигурява паралелна работа на всички функционални модули без компромиси в производителността или стабилността на системата.
- **Разширителни конектори**: стандартните конектори с 2.54 mm стъпка предоставят удобен и гъвкав начин за свързване на външни периферни модули, използвани в рамките на проекта. Платката предоставя разширителни конектори под формата на две 40-пинови рейки и една 60-пинова рейка, които извеждат директно I/O пиновете на FPGA и се използват за свързване на периферията.

Освен ресурсите, използвани пряко в разработката, платката разполага и с допълнителни хардуерни компоненти и интерфейси, които не участват в базовата реализация на системата, но формират част от цялостната хардуерна конфигурация на платформата. Тяхното наличие не оказва влияние върху архитектурните решения, разгледани в настоящата разработка, но подчертава универсалния характер на използваната развойна платка. Пример за такъв интерфейс е наличието на вграден видео изход (HDMI), който не се използва в базовата конфигурация на терминала, но е част от хардуерната функционалност на платформата.

В обобщение, QMTech Wukong Board V3 предоставя подходяща и балансирана хардуерна среда за реализиране на FPGA-базиран сериен терминал, като осигурява необходимите ресурси за текущата архитектура, без да въвежда излишна сложност или зависимост от специфични периферни компоненти.

2.6.2 Периферни интерфейси и физически ограничения

Развойната система налага строго дефинирани физически ограничения, които формират "рамката" на архитектурата:

1. **Тактов източник (System Clock):** Всички времеви диаграми в проекта са базирани на наличието на 50 MHz кварцов осцилатор, свързан към пин M21 на чипа. Това фиксира тактовата честота на системния домейн. За визуализационната подсистема се използва отделен пикселен такт 9 MHz, генериран чрез MMCM на базата на системния такт.
2. **Потребителски входове:**
    - **PS/2 Порт:** Платката осигурява хардуерна съвместимост със стандартни клавиатури чрез пинове, подсигурени с pull-up резистори.
    - **RS232-UART мост:** За комуникация с външни устройства (рутер/PC) се използва външен конвертор на нива (RS-232 трансивър), реализиран на дъщерна платка и свързан към пиновете на FPGA.
3. **Видео интерфейс:** За визуализация се използва 4.3-инчов TFT LCD панел (480x272), свързан чрез 40-пинов конектор. Архитектурата на изходния видео драйвер е проектирана да управлява директно паралелната RGB шина (24 бита) и управляващите сигнали (PCLK, DE, VSYNC, HSYNC), изведени на конкретни пинове на разширителните портове.

Тази хардуерна конфигурация дефинира граничните условия на проекта: системата трябва да се вмести в ресурсите на XC7A100T и да спазва електрическите стандарти на свързаната периферия.

2.6.3 Електрически стандарти и съгласуване на нивата

Един от критичните аспекти за успешната хардуерна реализация е съгласуването на напреженовите нива между ядрото на програмируемата логика и периферните устройства.

- **Входно-изходни банки:** Чипът работи с вътрешно напрежение на ядрото от 1.0 волта, но входно-изходните му блокове са конфигурируеми. В настоящия проект всички използвани банки са захранени с напрежение 3.3 волта.
- **Стандарт на сигналите:** Повечето сигнали в проекта (видео шина, клавиатурен вход) са конфигурирани в режим на нисковолтов CMOS стандарт (3.3V). Серийният RS-232 интерфейс е изключение и се реализира чрез отделен конвертор на нива. Това осигурява директна електрическа съвместимост с логическите нива на дисплея и клавиатурата, елиминирайки нуждата от допълнителни преобразуватели на напрежение.
- **Специализиран сериен интерфейс:** Тъй като стандартният протокол за серийна връзка работи с напрежения в диапазона ±12 волта, които не са съвместими с цифровите пинове на чипа, на дъщерната платка е интегриран специализиран приемник. Той преобразува високоволтовите сигнали от рутера в безопасни 3.3-волтови импулси за входа на системата. В комуникационния блок е използван RS-232 трансивър MAX3232, който работи при 3.3V и осигурява директна логическа съвместимост с FPGA. Сериен конектор DB9 (female) е реализиран така, че да осигури необходимото кръстосване на TX/RX (null-modem), което позволява свързване към DTE конзолен порт на мрежово устройство с прав кабел.

2.6.4 Конфигурация и енергонезависима памет

Важна особеност на избраната технология (базирана на SRAM клетки) е, че тя е енергозависима – при изключване на захранването конфигурацията на логическите връзки се губи.

За да функционира системата автономно като терминал, архитектурата включва използването на външна серийна флаш памет, разположена на платката. В използваната развойна платформа е налична SPI Flash памет с капацитет 32MB, използвана за съхранение на конфигурационния двоичен поток и постоянни ресурси. При подаване на захранване, чипът автоматично се установява в режим на главно устройство. Той инициира четене от флаш паметта, извлича конфигурационния двоичен поток и програмира вътрешната си матрица. Този механизъм позволява на терминала да стартира мигновено и да бъде готов за работа веднага след включване, без необходимост от връзка с компютър или програматор в работен режим.

2.6.5 Оценка на използваните ресурси

На етап концептуално проектиране е направена предварителна оценка на необходимите ресурси спрямо капацитета на избрания чип, за да се гарантира осъществимостта на проекта:

Памет: Използваният чип (XC7A100T) разполага с 135 физически блока памет, всеки с капацитет 36 килобита (общо 4,860 килобита). В таблица 5 са изложени изчисленията за необходимата ресурсна памет за помещаване на трите буфера необходими за реализацията на проекта.

Таблица 5 - Разпределение на ресурсите за блокова памет (BRAM)

Тип на буфера	Конфигурация (Дълбочина × Ширина)	Информационен обем	Определящ фактор за оразмеряването	Необходими физически блокове
Текстов буфер	17 адреса * 480 бита	8,160 бита	Ширина на шината	14
Знаков генератор	2048 адреса * 8 бита	16,384 бита	Капацитет Целият обем се побира свободно в един стандартен блок 16kb	1
Графичен буфер	4080 адреса * 32 бита	130,560 бита	Капацитет Общият обем изисква последователно свързване на няколко блока (130,560 / 36,000 ~ 3.6)	4
ОБЩО		**155,104 бита**		**19 блока**

Изчисленията показват, че за реализацията на текстовия буфер, знаковия генератор и графичния буфер са необходими около 20 блока памет. Това представлява под 15% от наличния капацитет на чипа (135 блока), което оставя значителен резерв. Текстовият буфер илюстрира интересна инженерна зависимост: въпреки че съдържа най-малко информация (само ~8 Kb), той изразходва най-много хардуерни ресурси (14 блока). Това се налага заради изискването за висока пропускателна способност – прочитане на цял ред за един такт, като реализацията на това изискване се

постига чрез паралелно свързване на множество блокове за едновременен достъп.

1. **Логически елементи:** Чипът разполага с 101,440 логически клетки. Проектираната система не използва сложни математически алгоритми (като цифрова обработка на сигнали) или софтуерен процесор. Основните потребители на логика са:
   - Броячи за видео синхронизация (хоризонтални и вертикални).
   - Крайни автомати за управление на данните (Data Handler).
   - Превключващи регистри за серийната комуникация (UART/PS2).

   Подобен тип логика традиционно заема между 1000 и 2000 клетки. Дори при най-песимистичната оценка (3000 клетки), заетостта ще бъде под 5%.

2. **Входно-изходни пинове:** Видео интерфейсът изисква най-голям брой физически връзки (за цветовите канали и синхронизацията) - общо 28 пина. Чипът разполага с над 200 потребителски пина, което прави опроводяването на периферията безпроблемно. Необходими са още 6 пина, необходими за свързването на клавиатурата, серийния интерфейс, кристала за тактовата честота и главен ресет.

2.7 Обобщение на архитектурните решения

В настоящата глава беше дефинирана архитектурната рамка на разработената FPGA-базирана система за сериен терминал. Проектирането е извършено на логическо и физическо ниво, като е поставен акцент върху паралелната обработка на данни, ясно разделение на функционалните отговорности и детерминирано поведение на системата.

Архитектурата следва принципите на поточния модел (dataflow architecture), при който входните данни, управлението на паметта и визуализацията се реализират като независими, но синхронизирани хардуерни подсистеми. Това разделение елиминира конкуренцията за ресурси и гарантира стабилна работа дори при високо натоварване от страна на входните интерфейси.

Йерархичната организация на паметта позволява ефективно преобразуване на данните от символно към пикселно представяне, като същевременно минимизира използването на вградени ресурси. Избраните архитектурни решения осигуряват пълно разделяне между процесите на генериране и визуализация на изображението, което е критично за надеждната работа на терминално устройство в реално време.

В допълнение, архитектурата е съобразена с физическите възможности на използваната развойна платформа и се вписва изцяло в наличните логически, паметни и входно-изходни ресурси. Описаните архитектурни принципи служат като основа за реализацията на RTL дизайна, който е разгледан подробно в следващата глава.

Глава 3 – Проектиране и изчисляване на системата

Настоящата глава разглежда реалната инженерна реализация на разработената система, като проследява прехода от архитектурните принципи, дефинирани в предходната глава, към конкретна цифрова и хардуерна имплементация. Реализацията на FPGA-базирана система по своята същност изисква едновременно описание на логическото поведение чрез хардуерно описание и осигуряване на физическа среда, в която това описание да функционира коректно.

Поради тази причина главата е структурирана в два основни раздела. Първият разглежда цифровата система, реализирана чрез регистърно-трансферно описание (RTL) на езика VHDL и синтезирана в програмируемата логика на FPGA чипа. Вторият раздел описва хардуерната реализация под формата на дъщерна платка, която осигурява електрическа съвместимост и физическо свързване с използваните периферни устройства. Тези два аспекта представляват неразделни части на една и съща инженерна реализация.

3.1 Цифрова система – RTL реализация

Цифровата система представлява логическото ядро на серийния терминал и е реализирана изцяло в рамките на FPGA матрицата. Всички функционални блокове – обработка на входни данни, управление на паметта и генериране на видео сигнал – са описани чрез RTL модели, което позволява паралелна работа, детерминирано времево поведение и ефективно използване на ресурсите.

Реализацията използва хардуерния описателен език VHDL и следва модулен подход. Всеки функционален блок е реализиран като самостоятелен RTL модул с ясно дефинирани интерфейси, което улеснява интеграцията, тестването и бъдещото разширение на системата.

За да се осигури ясна връзка между архитектурните решения, описани в предходната глава, и тяхната практическа реализация, цифровата система е разработена като набор от ясно разграничени RTL модули. Всеки модул покрива конкретна функционална част от системата и е реализиран в отделен VHDL файл или чрез конфигуриран IP блок.
В Таблица 6 е представен обзор на използваните файлове и тяхната роля в цялостната структура на проекта, като тази класификация служи като ориентир за последващото детайлно разглеждане на всеки модул.

Таблица 6 - Структура на RTL проекта и функционално покритие на модулите

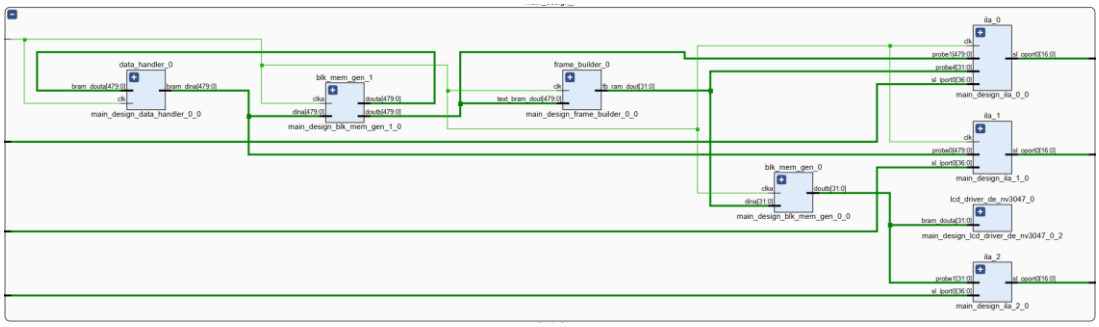
Файл / Модул	**Тип**	**Основна функция в системата**	**Тактов домейн**
main_design.v	Top-level RTL	Главен модул на системата. Дефинира външните интерфейси, инстанцира всички подмодули и осигурява свързването между тях.	50 MHz / 9 MHz
ps2_keyboard_to_ascii.vhd	RTL (Third-party)	Реализира PS/2 протокола за клавиатура и преобразува натиснатите клавиши в ASCII кодове с валидиращ сигнал.	50 MHz
uart_rx.vhd, uart_tx.vhd (Jakub Cabal)	RTL (Third-party)	Осигуряват асинхронна серийна комуникация (UART) и предоставят входно-изходен ASCII поток към системата.	50 MHz
data_handler.vhd	RTL (Авторска разработка)	Централен контролен модул. Обработва входните ASCII кодове, управлява курсора, скролиране и достъпа до	50 MHz

		текстовата памет.	
frame_builder.vhd	RTL (Авторска разработка)	Генерира графичното представяне на текста. Чете текстовия буфер и знаковия генератор и попълва видео буфера.	50 MHz
lcd_driver_de_nv3047_with_bram.vhd	RTL (Авторска разработка)	Управлява TFT LCD дисплея. Генерира видео тайминги и извежда пикселните данни към дисплея.	9 MHz
blk_mem_gen_1(Text Buffer)	IP (Xilinx BRAM)	Текстов буфер за съхранение на ASCII кодове, организирани по редове на екрана.	50 MHz
blk_mem_gen_2 (Font Memory)	IP (Xilinx BRAM)	Памет за шрифтове (8×16), инициализирана чрез .coe файл. Използва се само за четене.	50 MHz
blk_mem_gen_3 (Video Buffer)	IP (Xilinx BRAM)	Графичен видео буфер за съхранение на пикселните данни за целия екран.	50 MHz / 9 MHz
clk_wiz_0	IP (Xilinx Clocking Wizard)	Генерира производния пикселен такт (9 MHz) от системния такт (50 MHz).	50MHz

3.1.1 Обща структура на RTL дизайна

Главният RTL модул на системата е реализиран във файла main_design.v и представлява входната точка на целия цифров дизайн. Този модул дефинира външните интерфейси на серийния терминал, осигурява разпределението на тактовите сигнали и инстанцира всички функционални подмодули, необходими за реализиране на пълната функционалност на системата.

Ролята на top-level дизайна не е да реализира логика за обработка на данни, а да служи като структурен и свързващ слой, който обединява независимите RTL блокове в една работеща система. По този начин main_design.v представлява пряката връзка между архитектурния модел, описан в Глава 2, и конкретната реализация в програмируемата логика.



Фигура 6 - диаграма на top-level модула

Външни интерфейси на top-level модула

Top-level модулът дефинира всички физически входно-изходни сигнали, които свързват FPGA чипа с периферните устройства и развойната платка. Основните групи интерфейси включват:

- **Тактов и управляващ интерфейс**
  - вход за системен такт с честота 50 MHz, осигурен от кварцовия осцилатор на платката;
  - глобален reset сигнал за инициализация на всички подмодули.
- **Интерфейси за вход от потребителя**
  - PS/2 сигнали за комуникация с клавиатура;
- асинхронен сериен интерфейс (UART RX/TX) за връзка с външно мрежово оборудване.
- **Видео интерфейс**
  - паралелна RGB шина;
  - синхронизиращи сигнали (HSYNC, VSYNC, DE);
  - пикселен такт за управление на TFT LCD дисплея.

Тези интерфейси са дефинирани единствено в top-level модула и не се дублират в подмодулите, което осигурява ясно разделение между физическите входно-изходни сигнали и вътрешната логика на системата.

Таблица 7 - Външни портове на top-level модула

Име на сигнала	Посока	Ширина	Описание и предназначение
clk_50mhz	вход	1 бит	Основен системен такт с честота 50 MHz, осигурен от кварцовия осцилатор на развойната платка. Използва се като главен тактов домейн за цифровата логика.
reset_n	вход	1 бит	Глобален асинхронен reset сигнал (активен при логическа нула), използван за инициализация на всички вътрешни модули.
ps2_clk	вход	1 бит	Тактов сигнал на PS/2 интерфейса, използван за синхронна комуникация с клавиатурата.
ps2_data	вход	1 бит	Линия за данни на PS/2 интерфейса.
uart_rx	вход	1 бит	Входен сигнал за асинхронна серийна комуникация (UART Receive), използван за прием на данни от външно устройство.
uart_tx	изход	1 бит	Изходен сигнал за асинхронна серийна комуникация (UART Transmit), използван за предаване на данни към външно устройство.
lcd_r	изход	8 бита	Червен цветови канал на паралелната RGB видео шина към TFT LCD дисплея.
lcd_g	изход	8 бита	Зелен цветови канал на паралелната RGB видео шина към TFT LCD дисплея.
lcd_b	изход	8 бита	Син цветови канал на паралелната RGB видео шина към TFT LCD дисплея.
lcd_hsync	изход	1 бит	Хоризонтален синхронизиращ сигнал за LCD дисплея.
lcd_vsync	изход	1 бит	Вертикален синхронизиращ сигнал за LCD дисплея.
lcd_de	изход	1 бит	Data Enable сигнал, указващ валидните пикселни интервали за LCD дисплея.
lcd_pclk	изход	1 бит	Пикселен тактов сигнал за LCD дисплея, генериран чрез Clocking Wizard IP (9 MHz).

Инстанциране и свързване на функционалните модули

Във файла main_design.v са инстанцирани всички функционални RTL модули, необходими за реализацията на серийния терминал, като всеки модул изпълнява ясно дефинирана роля в системата. Top-level дизайнът осъществява тяхното логическо свързване и осигурява обмена на данни и управляващи сигнали между отделните функционални блокове.

Свързването между модулите е реализирано чрез вътрешни сигнали, които формират ясно разграничени интерфейси за пренос на данни, управление и синхронизация. Входните модули за клавиатура и сериен интерфейс предоставят ASCII кодове и валидиращи сигнали, които се насочват към централния модул за управление на данните. От своя страна той управлява достъпа до текстовия буфер и координира обновяването на съдържанието, което впоследствие се използва от генератора на кадри и видео подсистемата.

Особено внимание е отделено на разделянето на тактовите домейни при свързването на модулите. Логиката за обработка на входни данни, управление на паметта и генерация на кадри работи в системния тактов домейн с честота 50 MHz, докато LCD драйверът функционира в отделен видео домейн с пикселна честота 9 MHz. Преходът между тези домейни е реализиран чрез ясно дефинирани буфери и интерфейси, което гарантира коректна синхронизация и елиминира риска от времеви конфликти.

Поради сложността и йерархичния характер на цифровата система, пълната RTL диаграма на top-level дизайна, генерирана автоматично след синтеза на проекта, е изнесена в Приложение 6.

Там е представена детайлна визуализация на всички инстанцирани модули и техните взаимовръзки, която отразява реалната структура на синтезирания дизайн. В основния текст е акцентирано върху функционалните зависимости и принципите на свързване, без да се претоварва изложението с прекомерни структурни детайли.

По този начин top-level дизайнът изпълнява ролята на интеграционен слой, който обединява независимите RTL блокове в една цялостна и синхронизирана система, съответстваща на архитектурния модел, дефиниран в предходната глава.

Тактова организация в top-level дизайна

Top-level модулът осигурява разпределението на тактовите сигнали към отделните подмодули. Системният такт от 50 MHz се използва като основен тактов домейн за логиката по обработка на входните данни, управление на паметта и генерация на кадри.

Чрез инстанциране на IP блока Clocking Wizard се генерира производен тактов сигнал с честота 9 MHz, използван изключително от видео подсистемата и LCD драйвера. Разделянето на тактовите домейни е реализирано на top-level ниво, което гарантира, че всеки подмодул работи в коректния времеви контекст.

Инициализация и reset логика

Глобалният reset сигнал се разпределя от top-level модула към всички функционални блокове. При активиране на reset-а всички вътрешни регистри се инициализират в дефинирано начално състояние, което осигурява детерминирано стартиране на системата.

Особено важно е, че паметните блокове, използвани за текстовия буфер, видео буфера и знаковия генератор, са конфигурирани така, че при стартиране да съдържат валидни

начални стойности. Това позволява системата да започне работа веднага след конфигурацията на FPGA чипа, без необходимост от допълнителна софтуерна инициализация.

Роля на top-level дизайна в цялостната система

Най-високото йерархично ниво main_design.v изпълнява ролята на структурен гръбнак на цифровата система. Чрез него се реализира физическата и логическата интеграция на всички модули, като същевременно се запазва модулността и ясното разделение на отговорностите.

Този подход позволява всяка функционална част от системата да бъде анализирана и валидирана самостоятелно, което е от съществено значение за мащабируемостта, надеждността и бъдещото развитие на проекта.

3.1.2 PS/2 интерфейс (ps2_keyboard_to_ascii.vhd)

Комуникацията с потребителската клавиатура е реализирана чрез готов RTL модул за PS/2 интерфейс, разработен и публикуван от Digi-Key. Модулът реализира пълния PS/2 протокол на хардуерно ниво и осигурява надеждно декодиране на данните, предавани от стандартна PS/2 клавиатура.

В рамките на настоящия проект модулът се използва без функционални модификации, като е интегриран директно в top-level дизайна. Неговата роля е да преобразува физическите PS/2 сигнали (тактов сигнал и линия за данни) в унифициран символен интерфейс, базиран на ASCII кодове, който се използва от по-високите логически нива на системата.

Функционалност на PS/2 модула

PS/2 модулът изпълнява следните основни функции:

- синхронизация и филтриране на входните PS/2 сигнали;

- последователно приемане на данните, предавани от клавиатурата;
- декодиране на PS/2 сканкодове;
- преобразуване на получените кодове в ASCII формат;
- генериране на валидиращ сигнал, указващ наличие на нов символ.

По този начин сложността на PS/2 протокола е напълно изолирана от останалата част на цифровата система. Вътрешната логика на терминала работи единствено със символи, без да се влияе от особеностите на конкретния входен интерфейс.

Интерфейс и интеграция в системата

PS/2 модулът приема като входове тактовия сигнал и линията за данни от клавиатурата, както и системния такт на FPGA. На изхода той предоставя ASCII код на натиснатия клавиш, придружен от сигнал за валидност, който указва, че изходните данни са стабилни и готови за обработка.

Таблица 8 - Основни входове и изходи на ps2_keyboard_to_ascii.vhd

Име на сигнала	Посока	Ширина	Описание
clk	вход	1 бит	Системен тактов сигнал (50 MHz), използван за синхронизация на вътрешната логика на модула.
ps2_clk	вход	1 бит	Тактов сигнал на PS/2 интерфейса, идващ от клавиатурата.
ps2_data	вход	1 бит	Линия за данни на PS/2 интерфейса.
ascii_code	изход	7 бита	ASCII код на натиснатия клавиш, генериран след декодиране на PS/2 сканкодовете.
ascii_valid	изход	1 бит	Валидиращ сигнал, указващ наличие на нов, стабилен ASCII символ на изхода.
reset	вход	1 бит	Reset сигнал за инициализация на вътрешните регистри и състояния на модула.

Изходният ASCII интерфейс на PS/2 модула е свързан директно към централния модул за управление на данните

(data_handler.vhd). Това позволява клавиатурният вход да бъде третиран по същия начин както входните данни от серийния интерфейс, което е ключово архитектурно решение за унифициране на входните потоци.

Тактов домейн и времево поведение

PS/2 модулът работи в системния тактов домейн с честота 50 MHz, осигурен от развойната платка. Всички вътрешни операции по приемане и декодиране на данните са синхронизирани към този такт, което гарантира стабилна работа и коректна интеграция с останалите RTL модули.

Роля на PS/2 модула в цялостната реализация

Използването на готов PS/2 RTL модул позволява фокусиране на разработката върху специфичната функционалност на серийния терминал, без да се налага повторна реализация на добре дефиниран и стандартизиран протокол. В контекста на проекта този модул служи като надежден входен интерфейс, който предоставя символни данни към вътрешната логика и допринася за модулността и четимостта на цялостния дизайн.

3.1.3 UART интерфейс

Серийната комуникация с външно мрежово оборудване е реализирана чрез готов RTL модул за асинхронна серийна комуникация (UART), разработен от Jakub Cabal. Модулът осигурява надежден прием и предаване на данни в стандартен UART формат и е широко използван в FPGA-базирани системи поради своята ясно дефинирана архитектура и параметризируемост.

В рамките на настоящия проект UART модулът е интегриран като отделен функционален блок в top-level дизайна и работи в системния тактов домейн с честота 50 MHz. Основната му роля е да

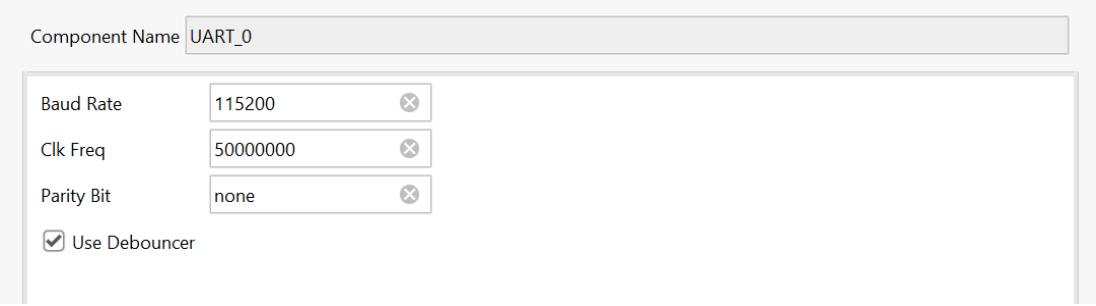
осигури комуникационен канал между серийния терминал и външно мрежово устройство, използвано за тестване и валидация на системата.

Конфигурация на UART модула

UART модулът е конфигуриран чрез параметри, зададени на етап интеграция в проекта, така че да отговаря на изискванията на избраното мрежово оборудване за тестване – рутер MikroTik RB433. Конфигурационните параметри са съгласувани със стандартните настройки за сериен конзолен достъп на устройството. Използваните настройки са:

- Скорост на предаване (Baud rate): 115200 bps
- Тактова честота: 50 MHz
- Parity bit: няма
- Формат на кадъра: 8 data bits, 1 stop bit (8N1)

Тази конфигурация осигурява пълна съвместимост със серийния конзолен интерфейс на рутера и позволява коректен обмен на ASCII символи без необходимост от допълнителна адаптация.



Фигура 7 - Конфигурация на UART модула в средата Vivado

Интерфейс и интеграция в системата

UART модулът предоставя разделени канали за прием (RX) и предаване (TX) на данни. Приетите символи се декодират и подават към вътрешната логика на системата под формата на ASCII кодове,

придружени от валидиращ сигнал. Аналогично, при предаване на данни, ASCII кодовете се сериализират и изпращат към външното устройство през TX линията.

Изходният ASCII интерфейс на UART модула е унифициран с този на PS/2 модула и е свързан директно към централния модул за управление на данните (data_handler.vhd). Това позволява входът от серийния интерфейс и входът от клавиатурата да бъдат обработвани по един и същ начин от останалата част на системата.

Тактов домейн и времево поведение

UART модулът работи изцяло в системния тактов домейн с честота 50 MHz. Вътрешната логика използва този такт за генериране на времевите интервали, необходими за асинхронната комуникация при зададената скорост от 115200 bps. По този начин се осигурява стабилна и детерминирана комуникация, синхронизирана с останалите модули в системата.

Таблица 9 - Основни входове и изходи на UART модула

Име на сигнала	Посока	Ширина	Описание
clk	вход	1 бит	Системен тактов сигнал с честота 50 MHz, използван за вътрешна синхронизация на UART модула.
reset	вход	1 бит	Reset сигнал за инициализация на вътрешните регистри и състояния.
uart_rx	вход	1 бит	Входна линия за асинхронна серийна комуникация (Receive).
uart_tx	изход	1 бит	Изходна линия за асинхронна серийна комуникация (Transmit).
rx_data	изход	8 бита	Приет ASCII символ, декодиран от серийния поток.
rx_valid	изход	1 бит	Валидиращ сигнал, указващ наличие на нов приет символ.
tx_data	вход	8 бита	ASCII символ за предаване към външното устройство.
tx_start	вход	1 бит	Управляващ сигнал за стартиране на предаване.

Роля на UART интерфейса в системата

UART интерфейсът осигурява основния комуникационен канал между серийния терминал и външното мрежово оборудване. Чрез него системата може да приема текстов изход от рутера и да предава команди, въведени от потребителя. Съгласуването на параметрите на UART модула с изискванията на избрания рутер гарантира надеждна работа на терминала в реален сценарий на използване.

3.1.4 Централен модул за управление на данните (data_handler.vhd)

Модулът data_handler.vhd представлява централното логическо ядро на цифровата система и реализира основната функционалност на серийния терминал. За разлика от входните и комуникационните модули, които са готови външни реализации, този модул е изцяло авторска разработка и отразява пряко архитектурните решения, дефинирани в предходната глава.

Основната роля на модула data_handler е да приема унифицирания символен поток (ASCII кодове), постъпващ от PS/2 и UART интерфейсите, да извършва семантична интерпретация на входните символи и да управлява актуализацията на текстовото съдържание, съхранено в текстовия буфер.

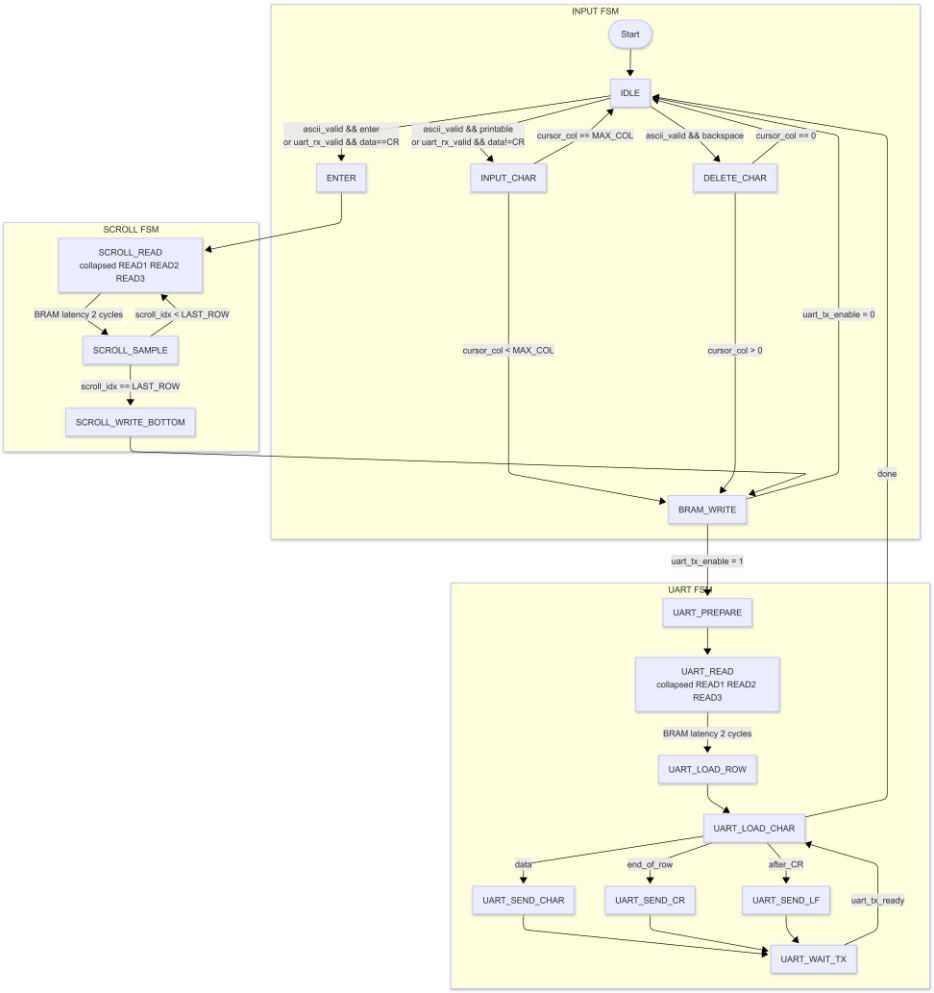
Функционална роля и вътрешна логика

Входните данни към модула data_handler.vhd се подават под формата на ASCII код и валидиращ сигнал, което позволява модулът да бъде напълно независим от конкретния източник на данните. Вътрешната логика е организирана около краен автомат на състоянията (FSM), който класифицира постъпващите символи и определя последващите действия.

Функционално входните символи се разделят на две основни категории:

- **Текстови символи**: стандартни печатаеми ASCII знаци, които се записват директно в текстовия буфер на позицията, определена от текущото състояние на курсора.
- **Управляващи символи**: специални кодове (например Enter и Backspace), които не се визуализират директно, а предизвикват промяна в позицията на курсора или активиране на допълнителна логика, като преминаване на нов ред, изтриване на символ или изпращане на данни по UART.

На Фигура 8 е представена диаграмата на крайния автомат, реализиран в модула data_handler.vhd. Крайният автомат управлява последователността от операции при обработка на входните символи, управлението на текстовия буфер, инициирането на серийно предаване и изпълнението на операции по скролиране. Диаграмата отразява реалните състояния, дефинирани в RTL описанието, и тяхната времева организация.



Фигура 8 - краен автомат на модула data_handler.vhd

Крайният автомат е реализиран като синхронен, многостъпков контролен автомат, работещ в системния тактов домейн. Всяко състояние съответства на конкретен етап от обработката, като сложните операции са разпределени в няколко последователни тактови цикъла с цел съобразяване с технологичните ограничения на използваните хардуерни ресурси.

- **Изчакване и първоначална обработка на входните символи**

Автоматът стартира и пребивава в състояние IDLE, в което изчаква постъпване на валидно входно събитие. Входното събитие се дефинира като наличието на ASCII символ, придружен от валидиращ сигнал, независимо от неговия произход – клавиатура (PS/2 интерфейс) или сериен интерфейс (UART).

При постъпване на такова събитие автоматът преминава в състояние INPUT_CHAR, където се извършва първоначална обработка и класификация на символа. В зависимост от стойността на ASCII кода се избира последващият път на изпълнение – запис на печатаем символ, изтриване или обработка на управляващ символ.

- **Обработка на печатаеми и управляващи символи**

При разпознаване на стандартен печатаем символ автоматът преминава към състояние BRAM_WRITE, в което се извършва запис на символа в текстовия буфер на позицията, определена от текущото състояние на курсора.

При получаване на управляващ символ за изтриване автоматът преминава в състояние DELETE_CHAR, където се реализира съответната логика за корекция на курсорната позиция и съдържанието на паметта.

- **Контекстно зависима обработка на символа Enter**

Състоянието ENTER има централна роля в логиката на терминала, тъй като в него се извършва разграничение според произхода на входното събитие.

Enter, получен от серийния интерфейс (UART):
В този случай символът се интерпретира като част от изходния текст, генериран от външното устройство. Крайният автомат подготвя преминаване на нов ред в текстовия буфер и, при необходимост, активира процедурата по скролиране, без да се инициира серийно предаване.

Enter, получен от клавиатурата (PS/2):
Тук символът има семантика на потребителска команда за изпращане на въведения ред. Крайният автомат преминава към

състояние UART_PREPARE, с което се стартира процедурата по серийно предаване на текущия ред към външното устройство.

Това контекстно разграничение позволява терминалът да функционира едновременно като входен интерфейс за команди и като изходно устройство за визуализация на данни.

- **Серийно предаване на текстов ред**

Процедурата по предаване на ред е реализирана чрез последователност от състояния, които осигуряват коректен достъп до паметта и синхронизация с UART модула:

- UART_PREPARE – инициализация на адреси и броячи;
- UART_READ1 → UART_READ2 → UART_READ3 – установяване на адрес, изчакване на латентността на паметта и подготовка за семплиране;
- UART_LOAD_ROW – зареждане на текущия ред;
- UART_LOAD_CHAR – извличане на отделен символ;
- UART_SEND_CHAR – подаване на символа към UART;
- UART_WAIT_TX – изчакване на завършване на предаването.

След предаване на всички символи от реда автоматът последователно изпраща управляващите символи Carriage Return (CR) и Line Feed (LF) чрез състоянията UART_SEND_CR и UART_SEND_LF, с което се осигурява съвместимост със стандартния терминален формат.

- **Скролиране на текстовия буфер**

При достигане на края на видимата област крайният автомат активира процедурата по скролиране, реализирана чрез състоянията:

SCROLL_READ1 → SCROLL_READ2 → SCROLL_READ3 – подаване на адрес и изчакване на латентността на паметта;

SCROLL_SAMPLE – семплиране на валидните данни и запис в по-горна позиция;

SCROLL_WRITE_BOTTOM – изчистване на последния ред.

Тези състояния реализират поетапно преместване на съдържанието на текстовия буфер с една позиция нагоре.

- **Технологични изчаквания при достъп до Block RAM**

Многоцикловите последователности от състояния при операциите *_READ1, *_READ2 и *_READ3 са въведени като технологична необходимост, произтичаща от времевия модел на използваната блокова памет (BRAM). При синхронните BRAM реализации валидните данни се появяват на изхода след определена латентност спрямо подаването на адреса. В използваната конфигурация е необходимо изчакване от два тактови цикъла, за да се гарантира коректно семплиране на данните. Крайният автомат отчита тази латентност чрез експлицитно разпределяне на операцията в последователни състояния.

- **Заключение**

Крайният автомат на модула data_handler реализира детайлно управлявана, детерминирана логика, която обединява обработката на входни символи, управлението на паметта, серийното предаване и визуализацията. Чрез ясно дефинирани състояния и технологично съобразени преходи се осигурява стабилна и предсказуема работа на серийния терминал при реални условия на използване.

Логика за управление на курсора и скролиране

Модулът поддържа текущата позиция на курсора чрез вътрешни регистри за хоризонтална и вертикална координата. При

запис на нов символ курсорът се измества автоматично, като се следят граничните условия на текущия ред и на видимата област на екрана.

Когато курсорът достигне края на последния текстов ред, data_handler генерира сигнал за активиране на механизма за скролиране. Скролиращата операция се реализира като последователност от контролирани операции за преместване на съдържанието в текстовия буфер, при което всички редове се изместват с една позиция нагоре, а последният ред се изчиства.

Тази логика е реализирана изцяло в хардуер и не изисква външен процесор или софтуерна намеса, което осигурява минимална латентност и детерминирано времево поведение.

Интерфейс към текстовия буфер

Модулът data_handler управлява записите в текстовия буфер чрез ясно дефиниран интерфейс за адресиране, данни и управляващи сигнали. Записът в паметта се извършва синхронно със системния такт и само при наличие на валиден входен символ или при активиране на операция по скролиране.

Тъй като текстовият буфер е споделен ресурс, достъпван и от други модули в системата, data_handler играе ролята на активен контролер на записите, като гарантира, че съдържанието на паметта винаги отразява текущото логическо състояние на терминала.

Тактова организация

Модулът data_handler работи изцяло в системния тактов домейн с честота 50 MHz. Всички вътрешни регистри, броячи и крайни автомати са синхронизирани към този такт, което осигурява коректна интеграция с входните интерфейси и паметните блокове.

Таблица 10 - Основни входове и изходи на модула data_handler.vhd

Име на сигнала	Посока	Ширина	Описание
clk	вход	1 бит	Системен тактов сигнал (50 MHz), използван за синхронизация на вътрешната логика.
reset	вход	1 бит	Reset сигнал за инициализация на регистрите и FSM състоянията.
ascii_in	вход	8 бита	Входен ASCII код, получен от PS/2 или UART модула.
ascii_valid	вход	1 бит	Валидиращ сигнал за наличие на нов входен символ.
mem_addr	изход	5 бита	Адрес към текстовия буфер, указващ позицията за запис.
mem_data	изход	480 бита	Данни за запис в текстовия буфер (ASCII код).
mem_we	изход	1 бит	Управляващ сигнал за разрешаване на запис в текстовия буфер.
scroll_req	изход	1 бит	Сигнал за заявка на операция по скролиране.

Роля на модула в цялостната реализация

Модулът data_handler.vhd реализира логиката, която превръща потока от входни символи в структурирано текстово съдържание. Чрез него се осъществява преходът от комуникационните интерфейси към визуализационната подсистема, което прави този модул ключов елемент за функционирането на серийния терминал.

3.1.5 Генератор на кадри (frame_builder.vhd)

Модулът frame_builder.vhd реализира преобразуването на символното съдържание, съхранено в текстовия буфер, в пикселно представяне, записано в графичния видео буфер. За разлика от концептуалното описание на визуализационния тракт в предходната глава, тук се разглежда конкретната RTL реализация, при която процесът на „рендериране“ се управлява от краен автомат, синхронизиран към системния такт.

Основната задача на frame_builder е да извърши следната последователност за всеки визуализиран символ и всеки пикселен ред от него:

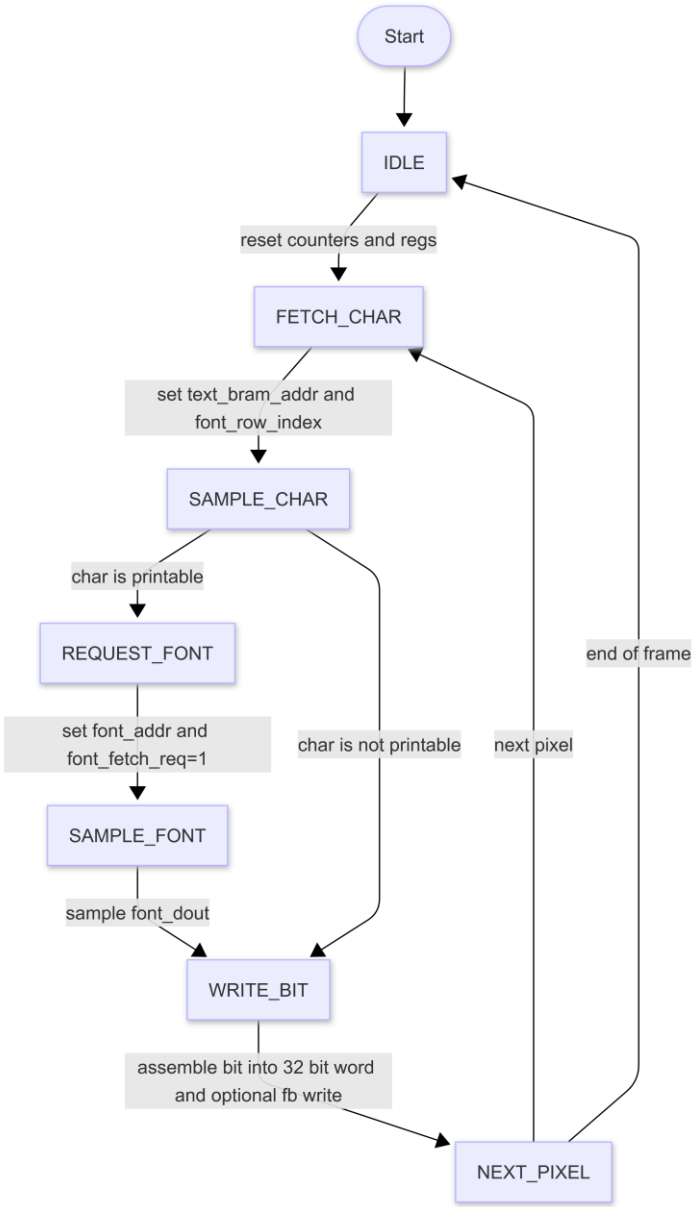
1. адресиране и прочит на ASCII символ от текстовия буфер;
2. формиране на адрес към шрифтовата памет (Font RAM/ROM) за конкретния ред на символа;
3. извличане на 8-битовата битова маска на реда;
4. запис на получените пикселни стойности към графичния буфер, като записът е организиран като последователност от операции „бит по бит“ и/или „пакетирано“ към 32-битова дума.

Краен автомат на модула и реална последователност на операциите

Функционирането на frame_builder се управлява от краен автомат със следните състояния:

IDLE, FETCH_CHAR, WAIT_T1, WAIT_T2, SAMPLE_CHAR, REQUEST_FONT, WAIT_F1, WAIT_F2, SAMPLE_FONT, WRITE_BIT, NEXT_PIXEL.

Тази структура е пряко следствие от използването на синхронни памети (Block RAM), при които прочитът на данни не е моментален и изисква технологична латентност.



Фигура 9 - краен автомат на модула frame_builder.vhd

1. Изчакване и заявяване на символ от текстовия буфер

Крайният автомат пребивава в IDLE, когато няма активна операция по генериране или когато е в междинно изчакване. При стартиране/продължаване на процеса автоматът преминава в FETCH_CHAR, където подава адрес към текстовия буфер за прочит на следващия ASCII символ.

След това следват състоянията WAIT_T1 и WAIT_T2, които представляват технологични изчаквания, необходими за валидиране

на данните на изхода на BRAM. Едва след изтичане на тези цикли автоматът преминава в SAMPLE_CHAR, където семплира стабилния ASCII код.

2. Заявяване на данни от шрифтовата памет

След прочита на ASCII символа автоматът формира адрес към шрифтовата памет, който обединява:

- ASCII кода (индекс на символа);
- индекса на текущия ред от матрицата на символа (0…15).

В състояние REQUEST_FONT този адрес се подава към Font RAM/ROM, след което отново се изчакват необходимите технологични цикли в WAIT_F1 и WAIT_F2. В SAMPLE_FONT се семплира валидната 8-битова стойност, представляваща пикселната маска на конкретния ред от символа.

3. Генериране и запис на пикселите

Състоянието WRITE_BIT реализира преобразуването на отделните битове от шрифтовата маска в пикселни стойности и подготовката им за запис в графичния буфер. В зависимост от конкретната организация на видео буфера (пакетиране по 32 бита) модулът акумулира последователни битове до формиране на пълна дума, след което извършва запис към BRAM (с активиране на write enable).

Състоянието NEXT_PIXEL инкрементира вътрешните индекси (хоризонтален пиксел, позиция на символа, пикселен ред в символа) и определя следващата стъпка:

- ако остава да се обработват пиксели в текущия символен ред → връщане към WRITE_BIT;
- ако трябва да се премине към следващ символ → FETCH_CHAR;

- ако трябва да се премине към следващ ред в символа → REQUEST_FONT;
- ако е достигнат край на активната област/кадъра → IDLE.

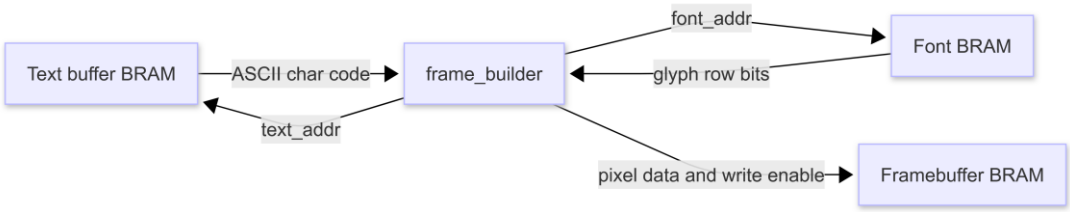
Технологични изчаквания при Block RAM достъп

Състоянията WAIT_T1, WAIT_T2 и WAIT_F1, WAIT_F2 са включени целенасочено като част от крайния автомат поради времевия модел на използваните BRAM блокове. При синхронен прочит адресът се установява в един тактов цикъл, а валидните данни на изхода се появяват след определена латентност. В използваната конфигурация са предвидени **два тактови цикъла**, които гарантират, че при семплиране в `SAMPLE_CHAR` и `SAMPLE_FONT` данните са коректни и стабилни. Този подход осигурява детерминирано поведение и елиминира риска от използване на междинни/невалидни стойности при рендериране.

Интерфейси към паметите и видео буфера

Модулът frame_builder реализира едновременно взаимодействие с три основни паметни ресурса:

- **Текстов буфер (read path):** за извличане на ASCII кодове по адрес;
- **Шрифтова памет (read path):** за извличане на 8-битова маска на ред от символа;
- **Графичен буфер (write path):** за запис на получените пикселни данни последователно по адреси.



Фигура 10 - поток на данните в модула frame_builder.vhd

Така модулът изпълнява функцията на „хардуерен рендерер“, който използва символни данни за генериране на пикселно съдържание.

Таблица 11 - Основни входове и изходи на модула frame_builder.vhd

Име на сигнала	Посока	Ширина	Описание
clk	вход	1 бит	Системен тактов сигнал (50 MHz).
reset	вход	1 бит	Reset сигнал за инициализация на регистрите и крайния автомат.
text_addr	изход	N бита	Адрес към текстовия буфер за прочит на ASCII символ.
text_dout	вход	8 бита	Прочетен ASCII код от текстовия буфер.
font_addr	изход	11 бита	Адрес към шрифтовата памет (ASCII + индекс на ред 0…15).
font_dout	вход	8 бита	8-битова пикселна маска (ред от 8×16 символ).
vid_addr	изход	N бита	Адрес към графичния буфер за запис.
vid_din	изход	32 бита	Данни за запис в графичния буфер (пакетирани пикселни стойности).
vid_we	изход	1 бит	Разрешение за запис в графичния буфер.

Роля на модула в реализацията

Модулът frame_builder.vhd реализира ключовата трансформация от символен към пикселен домейн и е пряката връзка между логическото съдържание (текст) и физическата визуализация (пиксели). Реализацията чрез краен автомат, съобразен с латентността на BRAM, осигурява стабилна и предсказуема генерация на графичния буфер, който впоследствие се използва от LCD драйвера за непрекъснато опресняване на дисплея.

3.1.6 LCD драйвер (lcd_driver_de_nv3047_with_bram.vhd)

Модулът lcd_driver_de_nv3047_with_bram.vhd реализира физическия видео интерфейс между цифровата система и TFT LCD дисплея. Той представлява крайната стъпка от визуализационния тракт и е отговорен за извеждането на пикселните данни и съпътстващите управляващи сигнали към дисплея в строго определена времева последователност.

За разлика от модулите за обработка на данни и генериране на кадри, LCD драйверът не извършва логическа интерпретация или модификация на съдържанието. Неговата функция е изцяло времево-ориентирана и се свежда до коректно адресиране на графичния видео буфер и синхронизирано извеждане на вече подготвените пикселни данни към конкретния LCD панел.

Място на LCD драйвера в архитектурата

LCD драйверът функционира като потребител на графичния видео буфер, който е предварително попълнен от модула frame_builder. Докато генераторът на кадри работи в системния тактов домейн и произвежда графично съдържание, LCD драйверът работи в отделен видео тактов домейн и извършва единствено последователен прочит и извеждане на пикселните данни.

Това архитектурно разделение напълно развързва процесите на генериране и визуализация на изображението и гарантира стабилна работа на дисплея независимо от моментното натоварване на останалата логика.

Режим на работа и времева организация на видео сигнала

LCD драйверът е реализиран в режим DE-only (Data Enable only), при който активната област на изображението се дефинира изцяло чрез сигнала Data Enable (DE) и пикселния такт. В този

режим синхросигналите HSYNC и VSYNC не участват функционално в управлението на видеотайминга и са фиксирани на постоянно логическо ниво.

Работата на драйвера се управлява от времево-ориентиран краен автомат, синхронизиран към пикселния такт. Автоматът функционира детерминирано въз основа на вътрешни броячи и не реагира на външни събития или входни данни.

Основният принцип на работа се базира на два брояча:

- хоризонтален брояч, който отчита позицията на текущия пиксел в рамките на един ред;
- вертикален брояч, който отчита текущия ред в рамките на кадъра.

Комбинацията от тези броячи определя дали системата се намира в активната област на изображението или в празните (blanking) интервали. В зависимост от това крайният автомат управлява:

- активирането на сигнала Data Enable (DE);
- разрешаването на прочит от графичния видео буфер;
- извеждането на RGB данните към дисплея.

По този начин видео сигналът се формира като строго периодична времева последователност, която се повтаря за всеки кадър.

Прочит и извеждане на пикселните данни

По време на активната област на изображението, маркирана чрез активен сигнал DE, крайният автомат разрешава последователен прочит от графичния видео буфер. Буферът е адресиран линейно, като всеки адрес съдържа 32-битова дума, представляваща пакет от пикселни данни.

Вътрешна логика в драйвера извежда съответните цветови компоненти към паралелната RGB шина в синхрон с пикселния такт. Извън активната област RGB изходите остават в статично състояние, без да се подават валидни пикселни данни. Това предотвратява появата на визуални артефакти и осигурява стабилно изображение.

Конфигурируеми параметри и адаптация към дисплея

LCD драйверът е реализиран като параметризируем модул, което позволява настройка на видео таймингите според характеристиките на използвания дисплей. Параметрите, зададени при конфигурацията на модула, дефинират геометрията и времевите интервали на видео сигнала в DE-only режим.

Активна област:

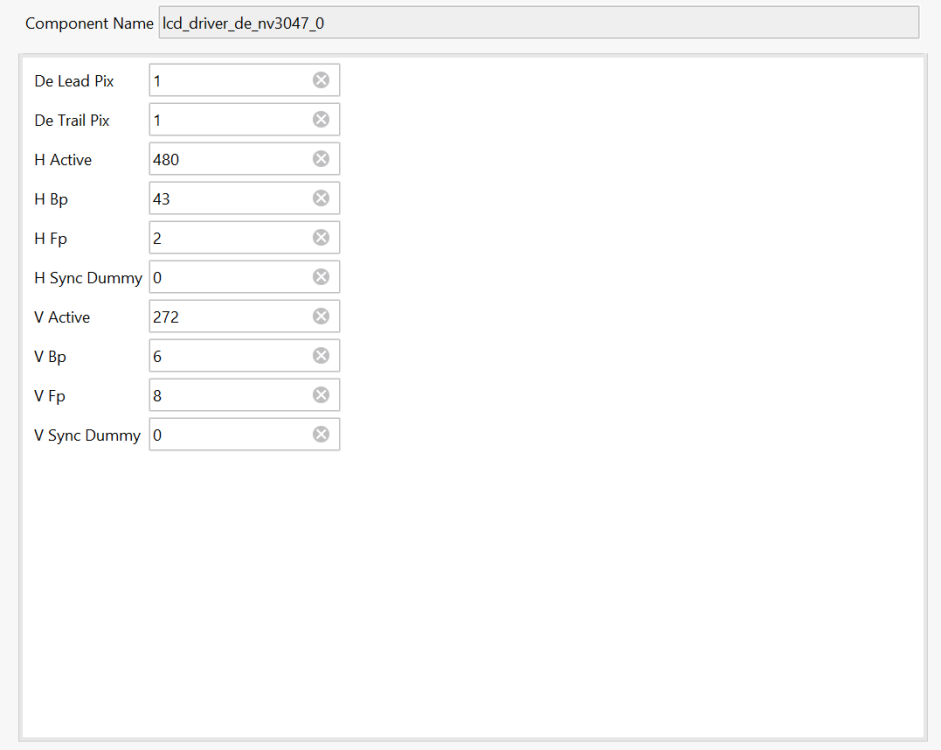
- H Active – брой активни пиксели в реда (480);
- V Active – брой активни редове в кадъра (272).

Тези параметри определят реалната видима резолюция на изображението.

DE интервали:

- DE Lead Pixels – брой пиксели преди активната област, при които сигналът DE се активира;
- DE Trail Pixels – брой пиксели след активната област, при които сигналът DE остава активен.

Тези параметри осигуряват точно времево съгласуване между RGB данните и сигнала DE и улесняват адаптацията към конкретния LCD панел.



Фигура 11 - Параметризация и времева организация на LCD драйвера

Тактова организация

LCD драйверът работи в отделен видео тактов домейн с пикселна честота 9 MHz, генерирана чрез IP ядро *Clocking Wizard*. Всички броячи, крайният автомат и операциите по прочит на графичния буфер са синхронизирани към този такт, което гарантира коректна времева връзка между данните и управляващите сигнали на дисплея.

Роля на модула в системата

LCD драйверът реализира директната връзка между цифровата логика и физическия дисплей. Чрез ясно дефинирана времева организация, DE-базиран видео режим и отделен тактов домейн се осигурява стабилна, детерминирана и надеждна визуализация. Това позволява серийният терминал да функционира като напълно самостоятелно хардуерно устройство, без

необходимост от софтуерно управление или външен видео контролер.

3.1.7 Паметни блокове (BRAM) и IP конфигурация

Блоковата памет (Block RAM, BRAM) представлява основния механизъм за съхранение и обмен на данни в разработената система. В рамките на проекта BRAM ресурсите на FPGA чипа се използват целенасочено за реализиране на три логически различни паметни области: текстов буфер, шрифтова памет и графичен видео буфер. Тези памети формират паметната основа на системата и осигуряват детерминиран и високопроизводителен достъп до данните.

Всички паметни блокове са реализирани чрез IP модула Block Memory Generator в средата за разработка Vivado, като конфигурацията им е съобразена с конкретните изисквания за достъп, пропускателна способност и времево поведение на отделните функционални модули.

Обща организация и роля на паметната подсистема

Паметната подсистема не представлява самостоятелен управляващ блок, а обобщава използваните BRAM ресурси и начина, по който те се интегрират в цифровата система. Управлението на достъпа до всяка памет се извършва от крайния автомат на съответния функционален модул (data_handler, frame_builder или lcd_driver), като по този начин се избягва необходимостта от централен контролер или сложна арбитражна логика. Това архитектурно решение позволява:

- ясно разграничение на отговорностите между модулите;
- детерминирано времево поведение;
- ефективно използване на BRAM ресурсите на FPGA чипа.

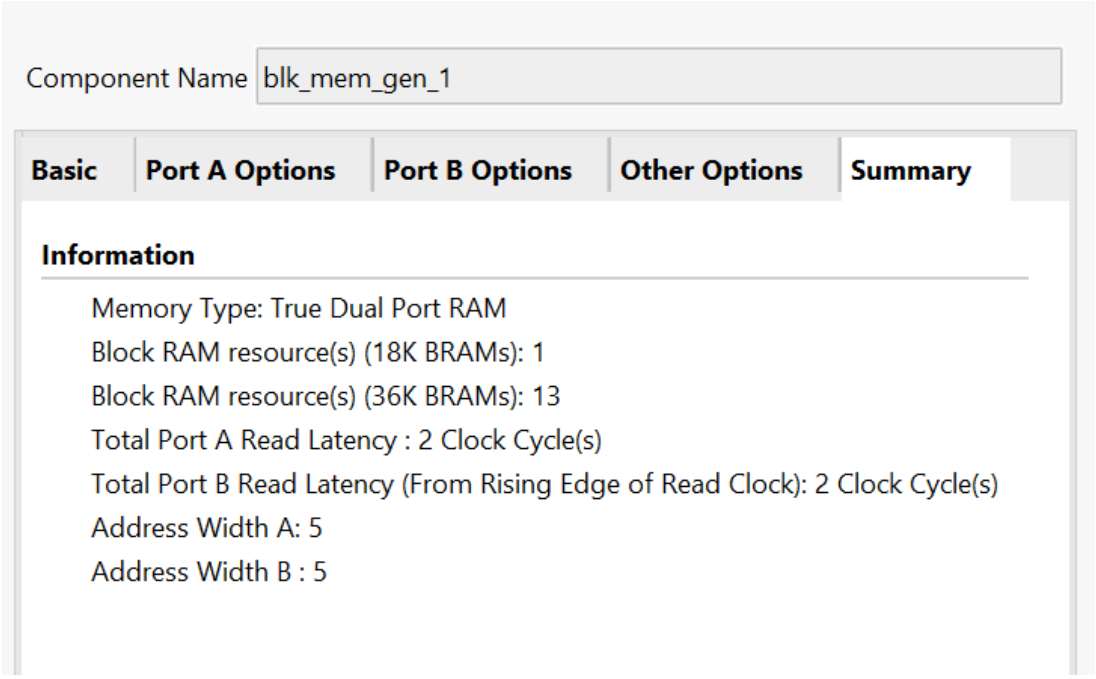
Текстов буфер

Текстовият буфер служи за съхранение на символното съдържание, което трябва да бъде визуализирано на дисплея. В него се записват ASCII кодовете, постъпващи от клавиатурата или серийния интерфейс, като организацията на паметта следва директно геометрията на екрана (редове и позиции в реда). Паметта е реализирана като True Dual Port RAM, което позволява:

- едновременен запис от модула data_handler;
- паралелен прочит от модула frame_builder.

Това е критично изискване за правилната работа на системата, тъй като въвеждането на нови символи и визуализацията на вече съществуващото съдържание се извършват паралелно.

На Фигура 12 е показана конфигурацията на текстовия буфер в средата Vivado.



Фигура 12 - Конфигурация на блокова памет (Text Buffer) в средата Vivado

Шрифтова памет

Шрифтовата памет съдържа графичните дефиниции на използваните символи и е реализирана като памет само за четене. Тя се инициализира при стартиране на системата чрез .coe файл, зареден в IP блока на блоковата памет. Всеки символ е представен като матрица от 8×16 пиксела, което позволява компактно и еднозначно преобразуване от ASCII код към пикселно представяне.

Достъпът до шрифтовата памет се осъществява ексклузивно от модула frame_builder, като адресът се формира чрез конкатенация на ASCII кода и индекса на текущия пикселен ред. Това осигурява директна и бърза транслация от символен към графичен домейн.

Графичен видео буфер

Графичният видео буфер представлява последното ниво в паметната организация и съдържа пълното пикселно изображение на екрана. Данните са организирани линейно и пакетирани в 32-битови думи, което позволява ефективно използване на BRAM ресурсите и последователен прочит с фиксирана скорост.

Достъпът до видео буфера е архитектурно разделен:

- **запис** – от модула frame_builder в системния тактов домейн;
- **четене** – от LCD драйвера в видео тактовия домейн.

Това развързване гарантира, че визуализацията остава стабилна и независима от процеса на генериране на кадъра.

Таблица 12 - Използвани паметни блокове и предназначение

Паметен блок	Тип памет	Основни модули	Предназначение
Текстов буфер	True Dual Port RAM	data_handler, frame_builder	Съхранение на ASCII кодове по редове
Шрифтов буфер	ROM (BRAM)	frame_builder	Графични дефиниции на символи (8×16)
Видео буфер	True Dual Port RAM	frame_builder, lcd_driver	Пълно пикселно изображение

Латентност при достъп до блоковата памет

Всички използвани BRAM блокове са конфигурирани като **синхронни памети с латентност на прочита от два тактови цикъла**, което означава, че валидните данни се появяват на изхода два такта след подаване на адреса. Тази характеристика е ясно видима в конфигурацията на IP блоковете, както е показано на приложените скрийншотове от средата за разработка.

Латентността е отчетена изрично в RTL дизайна чрез въвеждане на междинни изчакващи състояния в крайните автомати на съответните модули:

- при операции по скролиране и серийно предаване в data_handler;
- при прочит на ASCII символи и шрифтови данни в frame_builder;
- при последователния прочит на пикселни данни в LCD драйвера.

Този подход осигурява коректно семплиране на данните и детерминирано времево поведение на системата.

Обобщение

Чрез използването на параметризирани IP блокове за блокова памет и ясно дефинирани правила за достъп е реализирана ефективна и надеждна паметна организация. Архитектурното

разделяне на ролите на отделните памети, съчетано с крайни автомати, съобразени с латентността на BRAM, гарантира стабилна работа на серийния терминал и позволява паралелно генериране и визуализация на изображението без конфликти и визуални артефакти.

3.1.8 Обобщение на RTL реализацията

В рамките на подглава 3.1 детайлно е разгледана цифровата реализация на серийния терминал, изпълнена изцяло върху програмируема логика. Представените модули обхващат пълния функционален път на данните – от приемане на входни символи, през управление на паметта и генериране на графично съдържание, до физическата визуализация върху TFT LCD дисплей.

Архитектурата на RTL дизайна е изградена като съвкупност от ясно разграничени функционални модули, всеки със строго дефинирана роля и отговорности. Управлението на сложните последователности от операции е реализирано чрез крайни автомати, които осигуряват детерминирано времево поведение и позволяват коректна работа при наличие на технологични латентности, характерни за използваните блокови памети.

Особено внимание е отделено на управлението на паметните ресурси. Чрез използване на Block RAM IP блокове и целенасочен избор на конфигурации (True Dual Port RAM, синхронен прочит с фиксирана латентност) е реализирана ефективна паметна организация, която позволява паралелна работа на отделните подсистеми без конфликт при достъп. Латентността на паметите е отчетена експлицитно в RTL логиката чрез междинни състояния в крайните автомати, което гарантира стабилност и предсказуемост на системата.

Разделянето на логиката по тактови домейни допълнително допринася за надеждността на дизайна. Генерирането на кадрите и обработката на входните данни се извършват в системния тактов домейн, докато визуализацията е реализирана в независим видео тактов домейн. Това архитектурно решение разделя процесите на обработка и визуализация и елиминира риска от визуални артефакти при високо натоварване.

Важна характеристика на разработения RTL дизайн е липсата на софтуерен процесор и операционна система. Цялата функционалност е реализирана директно в хардуер, което осигурява мигновено стартиране, минимална латентност и висока надеждност. Това прави системата подходяща за използване като самостоятелен сериен терминал за диагностика на мрежово оборудване.

В допълнение, RTL диаграмите на всички основни модули, генерирани след синтез на проекта, са включени в приложенията към настоящата работа. Те илюстрират реалната структурна организация на дизайна и потвърждават коректността на описаната архитектура.

С приключването на цифровата реализация се създава стабилна основа за физическата реализация на системата. В следващата подглава е разгледана хардуерната част на проекта, включваща проектирането и реализацията на дъщерната платка, електрическото съгласуване на интерфейсите и интеграцията с развойната платформа.

3.2 Хардуерна реализация – дъщерна платка

Цифровата система, реализирана върху програмируема логика и описана в предходната подглава, изисква физическа хардуерна реализация, която да осигури електрическа съвместимост,

механична стабилност и надеждна връзка с периферните устройства. За тази цел е проектирана и изработена специализирана дъщерна платка, която функционира като shield / HAT модул, монтиращ се директно върху използваната развойна платка.

Проектирането на дъщерната платка е извършено в пълно съответствие с документациите на всички използвани интегрални схеми, техническата документация на TFT LCD дисплея и хардуерната документация на развойната платка. Основната ѝ задача е да осигури физическия интерфейс между FPGA базираната система и външните периферни устройства, без да въвежда допълнителна програмируема логика или управляващи елементи.

3.2.1 Концепция за интеграция и механично изпълнение

Дъщерната платка е проектирана за директен монтаж върху развойната платка чрез стандартни пинови конектори с растер 2.54 mm. За реализиране на този тип интеграция е извършена механична модификация на развойната платка, при която фабрично монтираните 90-градусови конектори са заменени с прави (вертикални) конектори. Това решение осигурява надежден електрически контакт, позволява директен монтаж на модули тип надстройка и позволява компактна и стабилна пространствена конфигурация на системата.

В резултат на тази интеграция дъщерната платка функционира като надстройка (shield), която добавя всички необходими периферни интерфейси към FPGA базираната система, без да променя нейната основна архитектура.

Механичното разположение и габаритите на платката са съобразени с тези на развойната платка, като монтажните отвори и конекторите са позиционирани така, че да се осигури съвместимост

и устойчивост при експлоатация. Реални изображения на реализираната печатна платка са представени в Приложение 10.

3.2.2 Функционални блокове на дъщерната платка

От функционална гледна точка дъщерната платка е организирана в няколко ясно разграничени хардуерни блока:

- интерфейс за серийна комуникация по стандарт RS-232;
- PS/2 интерфейс за клавиатура;
- TFT LCD интерфейс с паралелна RGB шина;
- схема за захранване и управление на LED подсветката на дисплея;
- резервиран USB интерфейс за бъдещо развитие.

Това функционално разделение позволява ясно проследяване на сигналните пътища и улеснява както анализа, така и бъдещи разширения на системата.

3.2.3 Интерфейс към TFT LCD дисплея

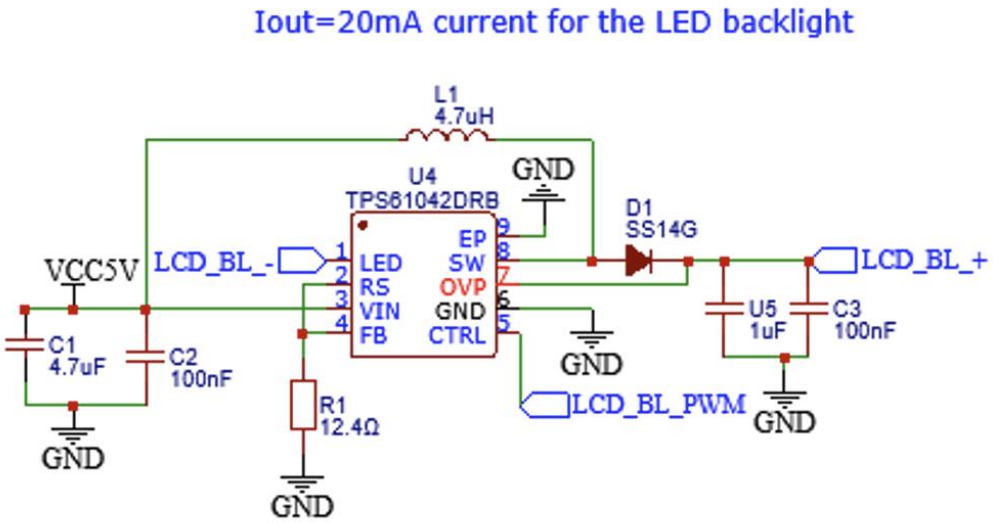
Свързването на TFT LCD дисплея е реализирано чрез 40-пинов FPC конектор, който осигурява всички необходими сигнали за паралелен RGB интерфейс, синхронизация и захранване. Цветовата информация е организирана в три осембитови шини (Red, Green и Blue), което позволява 24-битово цветово представяне.

Освен данните за цветовете, към дисплея се извеждат всички управляващи сигнали, включително пикселен такт, хоризонтална и вертикална синхронизация и сигнал Data Enable. В схемата са предвидени и допълнителни линии, резервирани за тъчскрийн функционалност, които не се използват в настоящата реализация, но позволяват бъдещо разширение.

3.2.4 Захранване на LED подсветката на дисплея

Захранването на LED подсветката на TFT дисплея е реализирано чрез специализирана схема за управление на светодиоди с постоянен ток. Схемата е оразмерена за номинален ток от 20 mA, което осигурява стабилна яркост при оптимална консумация на енергия.

В схемата е предвиден вход за PWM управление, което позволява регулиране на яркостта на подсветката директно от FPGA логиката. Това решение дава възможност за динамично управление на визуализацията и допринася за енергийната ефективност на системата.

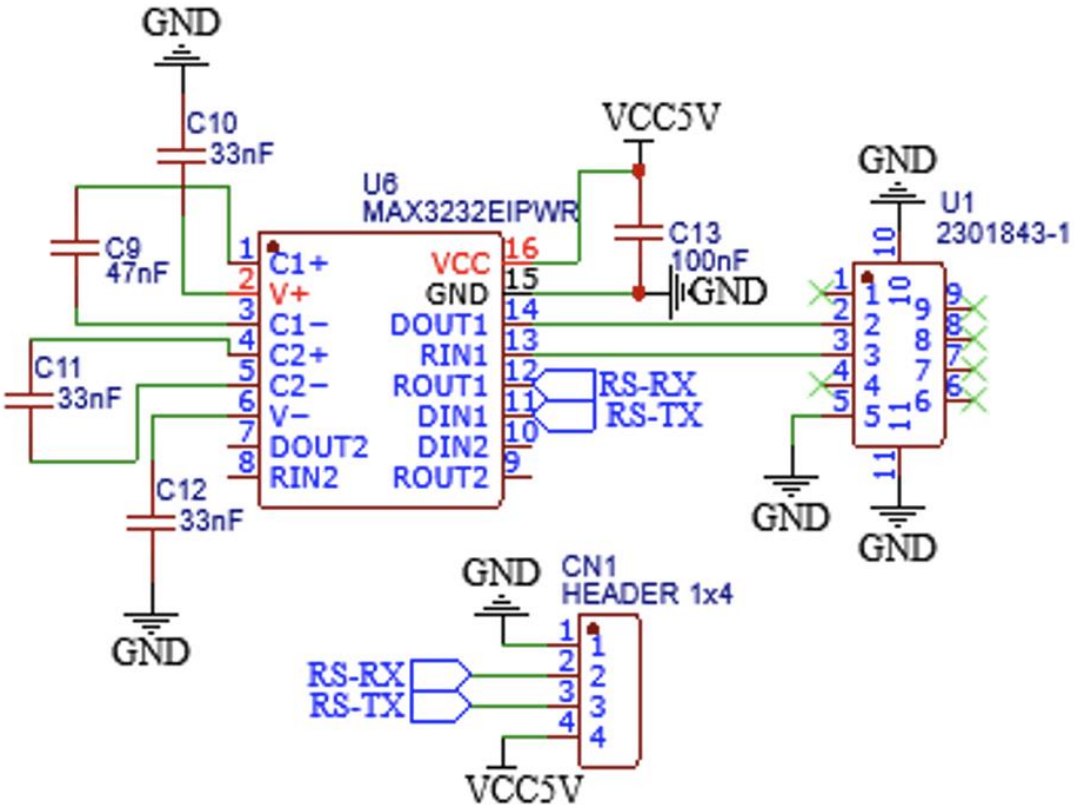


Фигура 13 - захранване на LED подсветката на дисплея

3.2.5 Сериен интерфейс RS-232

Серийната комуникация с външно мрежово оборудване е реализирана чрез стандарт RS-232. Поради несъвместимостта между логическите нива на FPGA чипа (3.3 V) и напреженовите нива на RS-232, в дъщерната платка е използван специализиран сериен приемник.

Този приемник осигурява преобразуване на нивата, както и защита на входно-изходните линии от пренапрежения. Интерфейсът е изведен към стандартен конектор, което позволява директна връзка с рутер или персонален компютър.



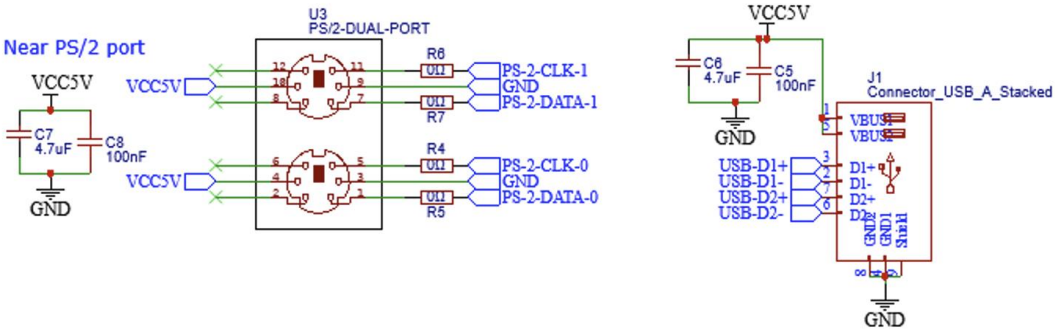
Фигура 14 - схема на RS-232 преобразувателя

3.2.6 PS/2 интерфейс и резервиран USB порт

За въвеждане на данни от потребителя е реализиран PS/2 интерфейс, който позволява директно свързване на стандартна клавиатура. В схемата са включени необходимите резистори и други пасивни елементи, разположени в близост до конектора, с цел минимизиране на смущенията и осигуряване на стабилна комуникация.

На дъщерната платка е предвиден и USB интерфейс, който не участва активно във функционалността на настоящата версия на

системата. Той е включен като хардуерен резерв за бъдещо развитие и потенциална интеграция на допълнителни периферни устройства.



Фигура 15 - схема на PS/2 и USB портовете

3.2.7 Реализация на печатната платка

Печатната платка е реализирана като двуслойна, с ясно разделение между сигнални трасета и маса. Трасировката е изпълнена с цел минимизиране на дължината на паралелните шини, групиране на високоскоростните сигнали и намаляване на електромагнитните смущения.

Горният и долният слой на платката и разположението на компонентите, са представени в Приложения 4 и 5. Там са показани както общият изглед на платката, така и детайлите на опроводяването.

3.2.8 Обобщение и препратка към приложенията

Разработената дъщерна платка осигурява пълната физическа реализация на FPGA базирания сериен терминал и позволява директната му интеграция с развойната платка под формата на shield/HAT модул. Чрез съобразяване с документациите на всички използвани компоненти, коректно съгласуване на електрическите нива и внимателно проектиране на печатната платка е постигната стабилна и надеждна хардуерна основа за работа на системата.

Пълните електрически схеми на дъщерната платка са включени в Приложения 1-3, а изображенията на горния и долния слой на печатната платка – в Приложения 4-5.

3.3 Оценка на използваните хардуерни ресурси

В настоящата подточка е направен детайлен анализ на използваните хардуерни ресурси на FPGA чипа XC7A100T, базиран на отчет за заетостта след имплементация (post-implementation utilization report), генериран от средата Vivado. Получените резултати са съпоставени с предварителните архитектурни изчисления, представени в Глава 2, с цел да се оцени ефективността на реализираната система и наличния ресурсен резерв.

Следва да се подчертае, че представените данни включват и диагностични модули (ILA), използвани единствено по време на разработката и тестването. Тези модули не са част от крайния функционален продукт и тяхното ресурсно потребление следва да бъде разглеждано отделно.

3.3.1 Логически ресурси (LUT и регистри)

След имплементация системата използва 7160 LUT от наличните 63 400, което съответства на 11.29% от общия логически капацитет. Използваните регистри (Flip-Flops) са 12 327 от 126 800, или 9.72%.

Анализът на модулно ниво показва, че значителна част от тези ресурси се дължат на интегрираните ILA модули, които добавят допълнителна логика за наблюдение на вътрешни сигнали. При изключване на диагностичната логика, ефективната заетост на LUT и регистрите за функционалната част на серийния терминал спада до приблизително 5–6%.

Тези стойности са в пълно съответствие с архитектурните очаквания, тъй като системата не използва изчислително интензивни алгоритми, а е изградена основно от крайни автомати, броячи и управляваща логика.

3.3.2 Използване на LUT като памет (LUTRAM)

Използването на LUT като памет възлиза на 1467 LUTRAM елемента, което представлява 7.72% от наличните ресурси. Това потребление е резултат основно от малки вътрешни буфери и регистрови структури, използвани за управление на състоянията и времевите последователности.

Този тип използване е очакван и не оказва негативно влияние върху наличността на стандартните LUT ресурси.

3.3.3 Използване на блокова памет (BRAM)

Блоковата памет е ключов ресурс за настоящия проект. След имплементация се отчита използване на 51 BRAM блока от общо 135, което съответства на 37.78% заетост.

В Глава 2 беше направено предварително изчисление, според което за текстовия буфер, знаковия генератор и графичния видео буфер са необходими приблизително 20 BRAM блока. Реално отчетената по-висока стойност се обяснява с факта, че ILA модулите използват допълнителна блокова памет за съхранение на проби от сигналите.

При елиминиране на диагностичните модули, използването на BRAM от функционалната част на системата остава близко до предварително изчислените стойности и заема значително под половината от наличния капацитет. Това потвърждава правилността на архитектурното решение за използване на вътрешна йерархия от памети.

3.3.4 DSP и специализирани ресурси

Системата използва 1 DSP блок от наличните 240, което представлява едва 0.42% от капацитета. Този минимален разход е индикация, че проектът не разчита на хардуерни умножители или цифрова обработка на сигнали, а е ориентиран изцяло към управляваща и комуникационна логика.

3.3.5 Тактови и входно-изходни ресурси

Използван е един PLL блок от наличните 6, което съответства на 16.67% заетост. PLL блокът се използва за генериране на пикселния такт за TFT дисплея, в съответствие с архитектурното решение за разделяне на тактовите домейни.

Входно-изходните ресурси са използвани в размер на 37 пина от общо 300, което представлява 12.33%. Най-голям принос за тази заетост има паралелният RGB интерфейс към дисплея, който изисква значителен брой линии за данни и синхронизация. Въпреки това остава значителен резерв от I/O ресурси.

3.3.6 Особености при отчитането на IP ядра

Следва да се отбележи, че използваните IP ядра на Xilinx (напр. Clocking Wizard и Block RAM Generator) не се визуализират като самостоятелни редове в отчета за заетост. Техните ресурси се добавят директно в съответните категории (BRAM, PLL, LUT и др.), което е стандартно поведение на средата Vivado.

Поради тази причина липсата на отделни редове за тези IP ядра не означава липса на ресурсна консумация, а е резултат от начина на отчитане на ресурсите.

3.3.7 Обобщение и инженерни изводи

Получените резултати от анализа на използваните хардуерни ресурси потвърждават пълното съответствие между архитектурните

изчисления и реалната имплементация на системата. Дори при включена диагностична логика, заетостта на основните ресурси остава ниска. След премахване на ILA модулите, предназначени единствено за разработка, функционалната част на серийния терминал използва минимална част от капацитета на FPGA чипа.

Това осигурява:

- стабилна и предсказуема работа на системата;
- значителен ресурсен резерв;
- възможност за бъдещо разширение без промяна на хардуерната платформа.

Глава 4 - Оценка на устройството и резултати

Настоящата глава представя резултатите от експерименталната проверка на разработената FPGA-базирана система за сериен терминал. Основната цел е да се потвърди коректността на архитектурните решения и реализацията, разгледани в предходните глави, чрез работа с реален хардуерен прототип и измервания в условия, близки до практическата експлоатация.

За разлика от чисто симулационния подход, в настоящата работа е възприета комбинирана методология, включваща вътрешна логическа верификация и физически електрически измервания. Вътрешното поведение на цифровата система се наблюдава чрез вграден логически анализатор (Integrated Logic Analyzer – ILA), докато реалните електрически сигнали по входно-изходните интерфейси се изследват с външна измервателна апаратура. Това позволява директна времева и функционална съпоставка между очакваното логическо поведение и реално наблюдаваните сигнали.

Изследванията са структурирани по функционални подсистеми, като за всяка от тях се разглеждат:

- входните и изходните сигнали;
- реакцията на вътрешните крайни автомати;
- взаимодействието с паметта и периферните интерфейси;
- съответствието между симулационните очаквания и физическите измервания.

4.1 Тестова постановка

Физическата верификация на FPGA-базирания терминал се извършва върху напълно интегриран хардуерен прототип,

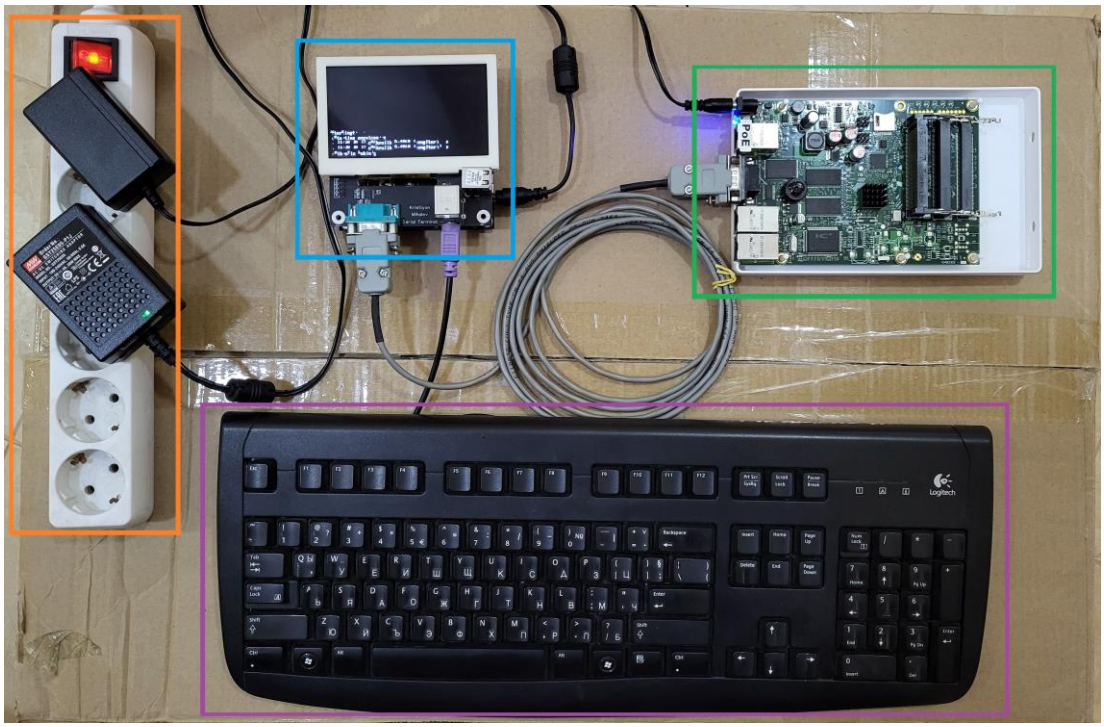
реализиран върху QMTech Wukong Board V3 (Xilinx Artix-7 XC7A100T) и свързан към TFT LCD дисплей с резолюция 480×272.

Тестовата постановка осигурява възможност за наблюдение на вътрешните събития в RTL логиката и на външните времедиаграми на LCD интерфейса при реална работа на системата.

4.1.1 Физическа конфигурация

Системата включва:

- FPGA развойна платка QMTech Wukong Board V3;
- дъщерна платка (shield/HAT) за периферни интерфейси;
- PS/2 клавиатура;
- TFT LCD дисплей 4.3", 480×272, TTL RGB интерфейс;
- захранващи блокове за логиката и дисплея.



Фигура 16 - Общ изглед на тестовата постановка

Оцветени са основните подсистеми: сериен терминал (синьо), мрежово оборудване (зелено), входно устройство – клавиатура (лилаво) и захранващи блокове (оранжево).

4.1.2 Измервателни средства

За целите на физическата верификация се използват следните измервателни средства:

Вграден логически анализатор (Integrated Logic Analyzer, ILA) – за наблюдение на вътрешни сигнали, адресни и информационни шини и контролни импулси в реален тактов домейн;

Осцилоскоп – за измерване на външни синхронизационни и диагностични сигнали.

Описаната тестова постановка служи като основа за всички последващи експериментални изследвания в настоящата глава. В следващите подточки се анализира поетапно поведението на входните интерфейси, вътрешната обработка на данните и визуализационната подсистема на системата.

4.2 Верификация на модула data_handler.vhd

Настоящото измерване има за цел експериментално да верифицира коректната работа на RTL модула data_handler.vhd, отговорен за обработката на ASCII входни данни, управлението на текстовия буфер и инициирането на операции по скролиране.

Верификацията се извършва върху реален хардуер чрез наблюдение на вътрешни сигнали с помощта на вграден логически анализатор (Integrated Logic Analyzer, ILA), като резултатите се съпоставят с очакваното поведение, дефинирано в архитектурата (Глава 2) и реализацията (Глава 3).

4.2.1 Цел на измерването

Целта на измерването е да се докаже, че модулът data_handler:

- коректно приема ASCII символи от входната подсистема;
- правилно адресира и записва данните в текстовия буфер;

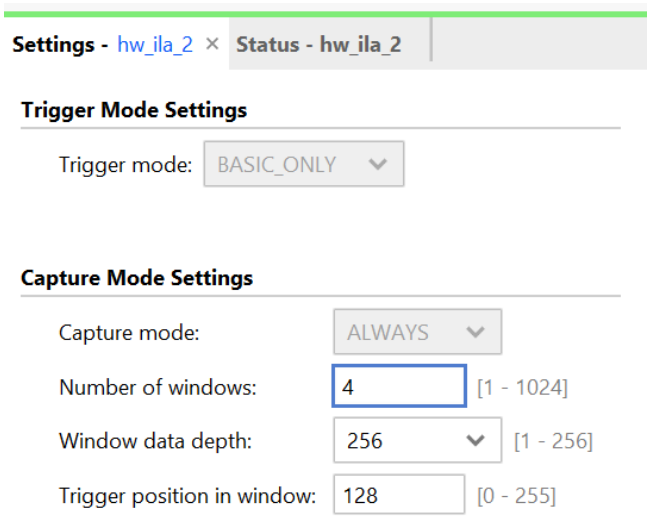
- реализира последователни burst операции към BRAM при специални управляващи събития (напр. нов ред);
- поддържа консистентно състояние на курсора и редовете в текстовия буфер.

Измерването е фокусирано върху динамичното поведение на модула при реални входни събития, а не върху статичната му структурна коректност.

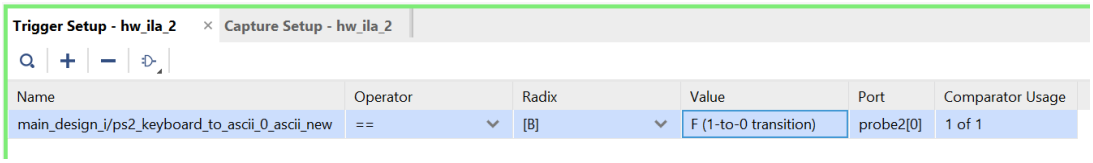
4.2.2 Методика на измерването

Методиката се базира на подаване на контролирани ASCII входни събития от PS/2 клавиатурата и наблюдение на вътрешните сигнали на data_handler чрез ILA.

Като тригерно условие за измерването се използва активиране на сигнал, индикиращ приемането на нов ASCII символ, което позволява точно времево подравняване на наблюдаваните данни и контролни импулси.



Фигура 17 - Конфигурация на ILA



Фигура 18 - Тригерно условие на ILA

4.2.3 Наблюдавани сигнали

За целите на анализа са наблюдавани следните вътрешни сигнали на модула:

- ascii_new – индикация за валиден входен ASCII символ;
- ascii_code[6:0] – ASCII код на приетия символ;
- bram_addra – адрес към текстовия буфер;
- bram_dina – данни за запис в BRAM;
- bram_wea – сигнал за разрешение на запис.

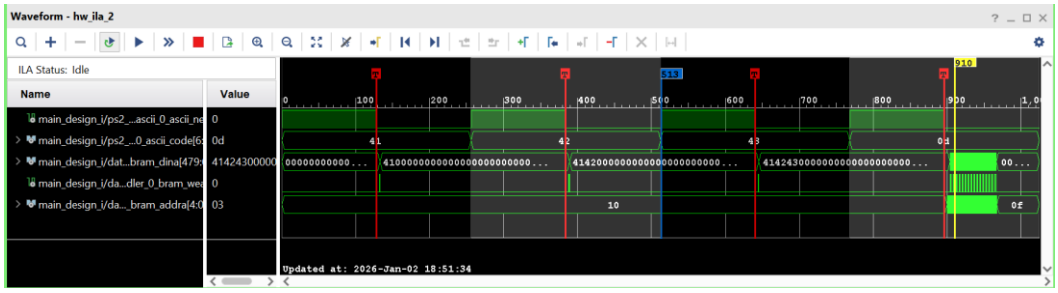
Този набор от сигнали позволява пълна реконструкция на последователността:

входно събитие → адресиране → запис в паметта.

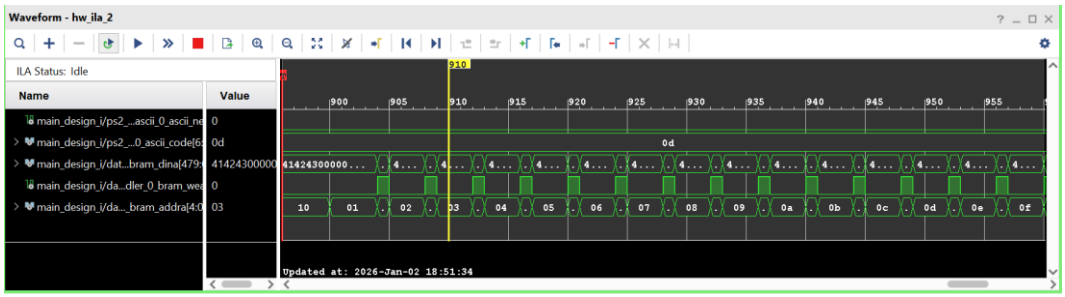
4.2.4 Резултати от измерването

При набиране на клавиатурата ASCII символите A,B,C, Enter (шеснадесетични кодове 0x41,0x42,0x43,0x0D) се наблюдава еднозначна корелация между импулса ascii_new, стойността на ascii_code и активирането на запис към текстовия буфер (bram_wea).

При специално управляващо събитие (нов ред - Enter) се регистрира серия от последователни записи с прогресиращ адрес bram_addra, което съответства на последователни операции по изместване на редовете в текстовия буфер (скролиране).



Фигура 19 - Общ изглед при работа на data_handler



Фигура 20 - Детайлен изглед на скролиращата операция

4.2.5 Анализ и интерпретация

Получените резултати потвърждават, че data_handler.vhd реализира коректно предвидената функционалност:

- ASCII символите се декодират и записват без загуба или припокриване;
- адресната логика на текстовия буфер функционира детерминирано;
- последователните операции при скролиране се изпълняват атомарно и без нарушаване на съдържанието на буфера.

Това измерване валидира модула като надежден входен етап за последващата визуализационна обработка, реализирана във frame_builder.vhd.

В рамките на настоящото измерване не се извършва отделна физическа верификация на UART приемника. UART модулът е реализиран като външен, вече изпитан RTL компонент и е използван в проекта без функционални модификации.

На входа на data_handler данните, получени от UART и от клавиатурната подсистема, са предварително нормализирани до общ ASCII интерфейс. В резултат на това логиката за обработка на входните символи, управление на текстовия буфер и иницииране на операции по скролиране е напълно преизползвана и независима от конкретния източник на данни.

Поради тази причина наблюдаваното поведение на data_handler при физическите ILA измервания служи не само като доказателство за коректната работа на модула при клавиатурен вход, но и като индиректно потвърждение за правилната функционална интеграция на серийния (UART) вход в рамките на системата.

4.3 Физическо измерване и верификация на UART предаване

4.3.1 Цел на измерването

Целта на настоящото измерване е да се верифицира коректното функциониране на UART предавателя (UART TX) като част от FPGA-базирания терминал, при предаване на текстов ред, въведен от клавиатурата. Проверява се, че:

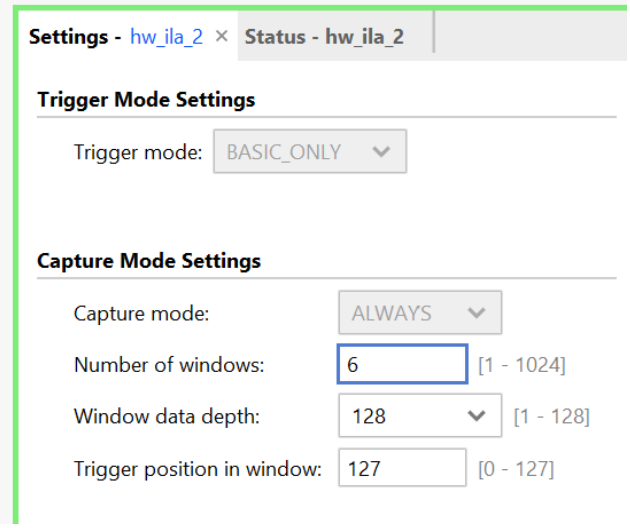
- въведените ASCII символи се буферират коректно до настъпване на събитие „край на ред“;
- при това събитие се формира изпращане на буферираните символи по UART;
- към предавания ред се добавят терминатори CR (Carriage Return, 0x0D) и LF (Line Feed, 0x0A), съгласно приетата терминална спецификация.

4.3.2 Методика на измерването

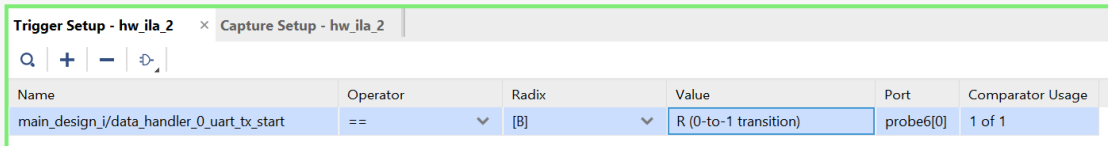
Като входен стимул се въвежда последователност от символи от PS/2 клавиатурата, последвана от управляващо събитие „нов

ред“. Вътрешното поведение се наблюдава чрез вграден логически анализатор (Integrated Logic Analyzer, ILA), конфигуриран да регистрира байтовете, подавани към UART предавателя, както и импулсите за стартиране на предаване.

Паралелно се изпълнява осцилоскопно измерване на физическата UART TX линия с активирана серийна декодировка (UART decode), което осигурява независима проверка на реално предадените символи.



Фигура 21 - Настройки на ILA за измерването на UART TX



Фигура 22 – Тригерно условие на ILA

4.3.3 Наблюдавани сигнали

За ILA измерването се наблюдават следните сигнали:

- ascii_new – индикация за валиден входен ASCII символ;
- ascii_code[6:0] – стойност на входния ASCII код;
- uart_tx_data[7:0] – байт, подаван към UART предавателя;

- uart_tx_start – импулс за стартиране на предаване на текущия байт.

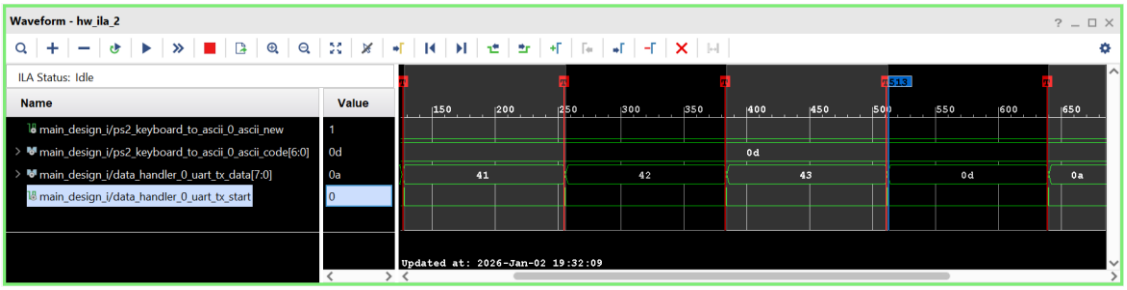
Този набор от сигнали позволява корелация между входния поток (клавиатура) и изходния поток (UART TX), както и идентификация на моментите на burst предаване.

4.3.4 Резултати от измерването

ILA измерването показва последователно подаване към UART предавателя на ASCII кодовете:

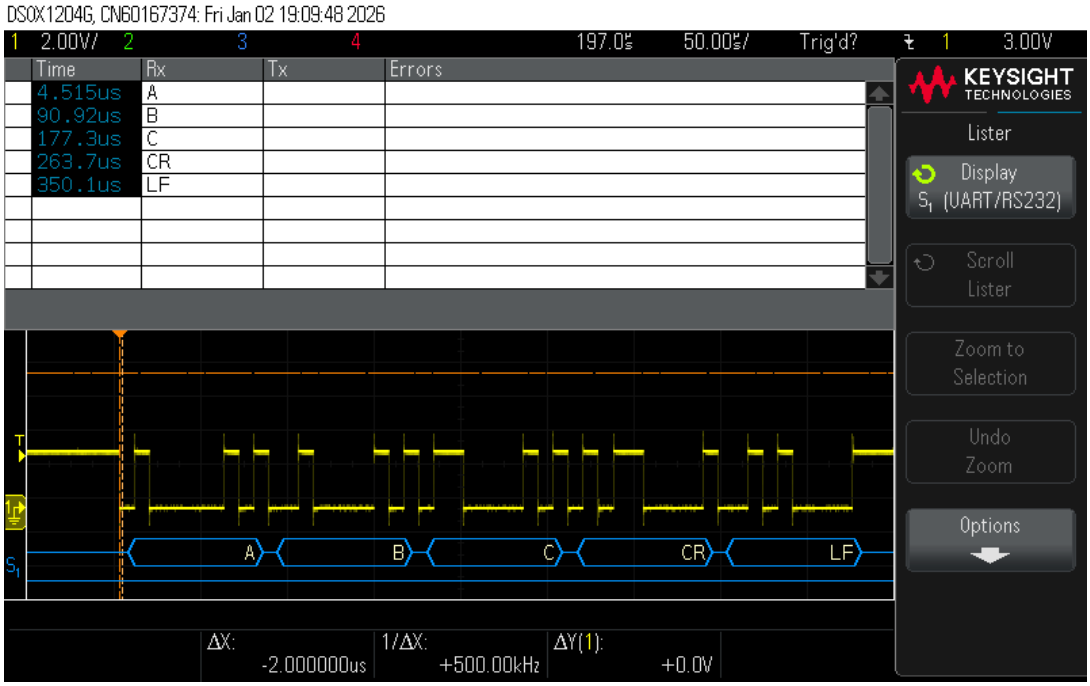
- 0x41, 0x42, 0x43 – съответстващи на символите A, B, C;
- 0x0D – CR (Carriage Return);
- 0x0A – LF (Line Feed).

Всеки байт е съпроводен от активиране на uart_tx_start, което потвърждава коректно стартиране на предаване за всяка позиция от burst последователността.



Фигура 23 - ILA измерване: UART TX

Осцилоскопната серийна декодировка на UART TX линията показва същата последователност от символи, предадени върху физическия интерфейс — **A, B, C, CR, LF**.



Фигура 24 - Осцилоскопно измерване: UART decode на TX линията

4.3.5 Анализ и интерпретация

Съвпадението между вътрешните ILA наблюдения и осцилоскопната декодировка на физическия сигнал доказва, че системата реализира коректно предаване на текстови редове към UART интерфейса. Добавянето на CR/LF след въвеждане на ред е в съответствие с очакваното поведение на сериен терминал и осигурява съвместимост със стандартни терминални приемници.

Настоящото измерване валидира както вътрешната логика за формиране на UART последователност, така и реалното сериализиране и предаване върху TX линията.

4.4 Физическо измерване и верификация на модула frame_builder.vhd

4.4.1 Цел на измерването

Целта на настоящото измерване е експериментално да се верифицира коректната работа на модула frame_builder.vhd при реална хардуерна експлоатация. Модулът реализира ключовата трансформация от символно представяне (ASCII кодове, съхранени

в текстовия буфер) към пикселно представяне, записано в графичния видео буфер.

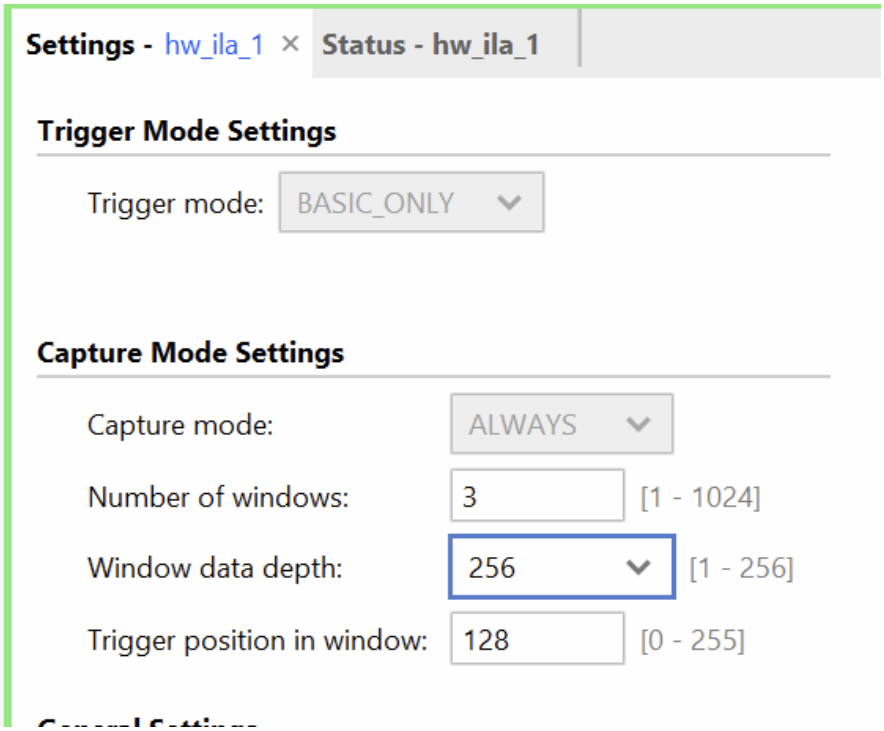
Измерването има за цел да докаже, че:

- frame_builder извършва детерминирана обработка на данните от текстовия буфер;
- адресирането и записът към графичния буфер се осъществяват коректно;
- генерираните записи съответстват на реално формирано графично съдържание, а не на празни или неинициализирани данни.

4.4.2 Методика на измерването

Измерването е извършено върху реална FPGA система чрез **вграден логически анализатор (Integrated Logic Analyzer, ILA)**. За разлика от симулационните тестове, този подход позволява наблюдение на реалните вътрешни сигнали при реална работа на дисплея и всички подсистеми.

ILA е конфигуриран в режим на непрекъснато захващане (continuous capture), като тригерирането е обвързано директно с активността на графичния буфер. По този начин се гарантира, че захванатите данни отразяват реални операции по рендериране, а не фонови или неактивни състояния.

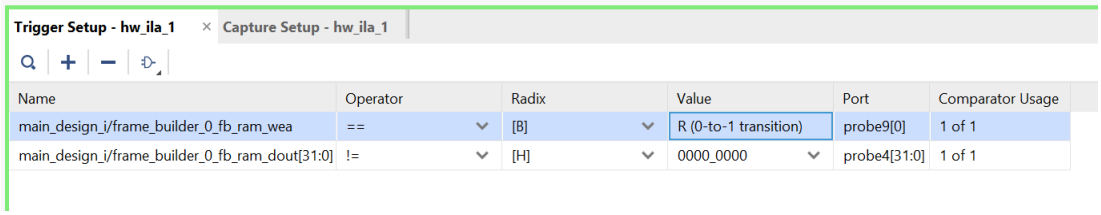


Фигура 25 - Настройки на ILA за измерването на frame_builder

4.4.3 Конфигурация на тригера и наблюдавани сигнали

За осигуряване на максимална информативност тригерът на ILA е дефиниран чрез логическа комбинация от два сигнала:

- frame_builder_0_fb_ram_wea – детекция на преход от 0 към 1 (Rising Edge), указващ начало на запис към графичния буфер;
- frame_builder_0_fb_ram_dout[31:0] != 0x00000000 – условие, гарантиращо, че записваните данни не са нулеви. Тази комбинация е избрана с цел визуално откроряване на реални операции по рендериране, тъй като през по-голямата част от времето графичният буфер съдържа нулеви данни. По този начин захващането се фокусира върху периодите с действително генерирани пикселни стойности.



Фигура 26 - ILA тригерно условие за frame_builder

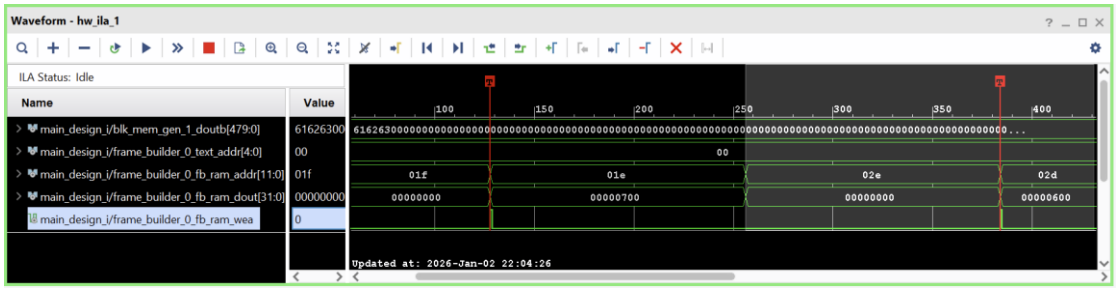
В рамките на измерването са наблюдавани следните вътрешни сигнали:

- blk_mem_gen_1_doutb[479:0] – изходни данни от текстовия буфер;
- frame_builder_0_text_addr[4:0] – адрес на текущия текстов ред;
- frame_builder_0_fb_ram_addr[11:0] – адрес към графичния видео буфер;
- frame_builder_0_fb_ram_dout[31:0] – данни за запис в графичния буфер;
- frame_builder_0_fb_ram_wea – разрешение за запис.

Този набор от сигнали позволява проследяване на пълната причинно-следствена верига от текстова информация към пикселни данни.

4.4.4 Резултати от измерването и анализ

На Фигура 27 е представен общ ILA изглед от измерването, в който се наблюдава ясно структурирана последователност от операции, характерна за работата на frame_builder.



Фигура 27 - ILA измерване на работата на frame_builder

Анализът на сигналите показва следното:

- **Стабилност на текстовите данни**: Данните на шината blk_mem_gen_1_doutb остават стабилни за интервала, през който

frame_builder обработва съответния текстов ред. Това потвърждава, че модулът работи със статично, вече формирано текстово съдържание, генерирано от data_handler.

- **Последователна адресна прогресия**: Адресът frame_builder_0_fb_ram_addr се изменя последователно във времето, което е индикатор за линейно попълване на графичния буфер. Това поведение е в пълно съответствие с архитектурното решение за пакетиране на 32 пиксела в една 32-битова дума.
- **Генериране на ненулеви пикселни данни**: Данните на шината frame_builder_0_fb_ram_dout съдържат ненулеви стойности (например 0x00000700, 0x00000600), което показва, че записваните пиксели отразяват реални активни точки от глифовете, извлечени от знаковия генератор.
- **Контролирани записи към паметта**: Сигналът frame_builder_0_fb_ram_wea се активира синхронно с валидни адреси и данни, като ясно маркира началото и края на всеки burst от записи към графичния буфер.

Наблюдаваното поведение ясно демонстрира, че frame_builder изпълнява правилно ролята си на транслационен блок между текстовото и графичното ниво на системата.

4.4.5 Изводи от измерването

Резултатите от физическото измерване чрез ILA потвърждават, че модулът frame_builder.vhd функционира коректно при реална работа на системата. Налице е детерминирана и повторяема причинно-следствена последователност:

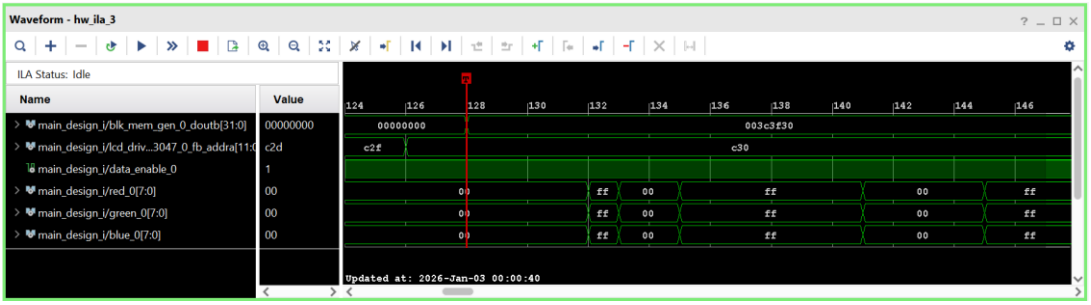
текстов буфер → графичен видео буфер → пикселни данни

По този начин експериментално се доказва, че графичното представяне на текстовата информация е реализирано изцяло в хардуер, без загуба на данни и без зависимост от софтуерна обработка. Това измерване затваря логическата верига между подсистемата за управление на данните и подсистемата за визуализация, като предоставя ключово доказателство за правилната архитектурна реализация на серийния терминал.

4.5 Физическо измерване и верификация на LCD драйвера

4.5.1 Вътрешно наблюдение чрез ILA

На Фигура 28 е показан произволен времеви фрагмент от работата на LCD драйвера, захванат чрез вграден логически анализатор (Integrated Logic Analyzer, ILA) по време на нормална експлоатация на системата. Измерването не е синхронизирано към конкретно събитие (напр. начало на кадър), а представя типично поведение на драйвера при извеждане на активни пикселни данни.



Фигура 28 - ILA измерване на работата на LCD драйвер

Наблюдаваните сигнали включват:

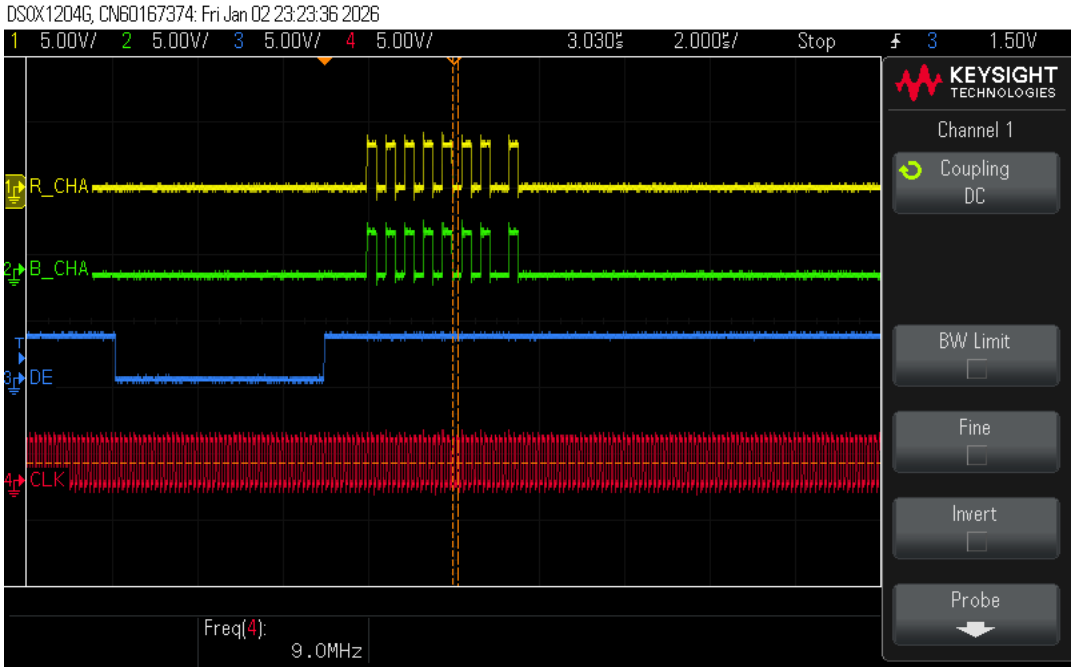
- изходни данни от графичния буфер (blk_mem_gen_0_doutb[31:0]);
- адресиране на графичния буфер (lcd_drv_3047_0_fb_addra);
- сигнал за активна видео област (data_enable_0);
- изходните цветови компоненти red[7:0], green[7:0] и blue[7:0].

В показания интервал сигналът data_enable_0 е активен, което индикира, че драйверът се намира в рамките на активната част на кадъра. Едновременно с това адресът към графичния буфер се изменя последователно, а на RGB изходите се наблюдават ненулеви стойности (напр. 0xFF), характерни за реално пикселно съдържание. Това потвърждава, че LCD драйверът извършва коректно растерно сканиране на видео буфера и формира цветови данни в синхрон с активната видео област.

Важно е да се отбележи, че представеният ILA кадър е случаен изрез от работата на драйвера, а не специално подбран сценарий. Това демонстрира стабилността и детерминираността на поведението при непрекъсната работа.

4.5.2 Външно физическо измерване чрез осцилоскоп

На Фигура 29 е показано осцилоскопно измерване на физическия LCD интерфейс, при което са наблюдавани сигналите пикселен такт (CLK), Data Enable (DE) и представителни линии от RGB шината. Измерването показва стабилен пикселен такт с честота приблизително 9 MHz, както и ясно дефиниран активен видео интервал, маркиран чрез сигнала DE.



Фигура 29 - осцилоскопно измерване на изходните сигнали, генерирани от LCD драйвера

По време на активния DE интервал се наблюдават преходи по RGB линиите, синхронни с пикселния такт, докато извън него цветните сигнали остават статични. Това поведение потвърждава, че драйверът извежда валидни пикселни данни единствено в рамките на активната част на кадъра.

4.5.3 Съпоставка и интерпретация

Комбинираното използване на ILA и осцилоскоп осигурява пълна верификация на LCD драйвера както на вътрешно, така и на физическо ниво. ILA измерването демонстрира коректното адресиране на графичния буфер и формирането на RGB данни в синхрон с активната видео област, докато осцилоскопното измерване потвърждава, че това поведение се реализира коректно върху реалния хардуерен интерфейс към дисплея.

4.5.4 Бележка относно HSYNC и VSYNC сигналите

В рамките на настоящото измерване сигналите HSYNC (Horizontal Sync) и VSYNC (Vertical Sync) не са изследвани нито

чрез ILA, нито чрез осцилоскоп. Причината е, че в реализирания LCD драйвер те не участват функционално в дефинирането на видеотайминга и са фиксирани на постоянно логическо ниво. Драйверът работи в режим DE-only (Data Enable only), при който активната видео област и растерното сканиране се определят изцяло чрез сигнала DE и пикселния такт.

Поради тази причина измерването на HSYNC и VSYNC не би добавило допълнителна информативна стойност към анализа, докато наблюдението на DE, пикселния такт и RGB линиите предоставя директно и достатъчно доказателство за коректната работа на драйвера.

4.5.5 Извод от измерването

Представените измервания доказват, че LCD драйверът реализира стабилен и детерминиран видео изход, при който пикселните данни се извеждат коректно в активната област на кадъра и са синхронизирани с пикселния такт. Това експериментално затваря веригата **графичен буфер → LCD драйвер → физически дисплей** и представлява финалната верификация на визуализационната подсистема.

4.6 Обобщение и изводи от физическите измервания

В настоящата глава е извършена експериментална верификация на основните функционални блокове на FPGA-базирания сериен терминал чрез измервания върху реален хардуер. Подходът комбинира вътрешно наблюдение с вграден логически анализатор (ILA) и външни физически измервания с осцилоскоп, което позволява цялостна оценка на поведението на системата в реални условия.

Последователно са верифицирани следните подсистеми:

- **Модулът data_handler**, при който е доказано коректното приемане на ASCII символи, управлението на текстовия буфер и инициирането на burst операции при скролиране;
- **UART предавателният път**, при който е потвърдено коректното формиране и предаване на текстови редове с терминатори CR/LF, съгласно терминалната спецификация;
- **Модулът frame_builder**, при който е експериментално доказана трансформацията от символно към пикселно представяне чрез последователни паметни транзакции;
- **LCD драйверът**, при който е потвърдено коректното адресиране на графичния видео буфер и извеждането на пикселни данни към дисплея в DE-only режим, синхронизирано с пикселния такт.

Наблюдаваните резултати показват детерминирано и стабилно поведение на системата, като всички измервания съвпадат с очакваното поведение, дефинирано в архитектурния модел (Глава 2) и реализацията (Глава 3). Използването на реален хардуер и изключването на ILA ресурсите от ресурсната оценка осигуряват реалистична и коректна експериментална основа.

По този начин се потвърждава, че разработената система реализира пълната функционалност на сериен терминал изцяло в хардуер, без необходимост от софтуерна обработка или външен видео контролер.

Заключение

В настоящата дипломна работа е разработен и верифициран FPGA-базиран сериен терминал с клавиатурен вход и LCD дисплей, реализиран върху платформа QMTech Wukong Board V3 с FPGA Xilinx Artix-7 XC7A100T. Системата интегрира входни интерфейси (PS/2 и UART), управление на текстови и графични данни и пълна визуализационна подсистема, реализирана изцяло в цифрова логика.

Проектът е изграден по top-down архитектурен подход, при който първоначално е дефинирана логическата структура на системата, последвана от детайлна RTL реализация и експериментална верификация. Основните функционални блокове са ясно разделени, като всеки от тях изпълнява строго дефинирана роля и комуникира с останалите чрез добре формализирани интерфейси.

Особен акцент в реализацията е използването на йерархия от памети, включваща текстов буфер, Font ROM и графичен видео буфер, както и реализирането на LCD драйвер в DE-only режим, адаптиран към конкретния TFT панел. Това архитектурно решение опростява видео интерфейса и позволява стабилна и детерминирана визуализация при минимална логическа сложност.

Експерименталната част на работата доказва, че системата функционира коректно в реални условия, като всички ключови процеси – от приемането на символи до физическото извеждане на пикселни данни – са валидирани чрез измервания върху реален хардуер. Получените резултати потвърждават съответствието между проектираната архитектура и реалната реализация.

Разработеното решение представлява завършен прототип, който демонстрира възможностите на FPGA технологиите за изграждане на самостоятелни интерактивни системи. Архитектурата и реализацията позволяват бъдещо разширение – например добавяне на цветни шрифтове, графични примитиви, по-висока резолюция или допълнителни комуникационни интерфейси – без фундаментални промени в структурата на системата.

С това целите на дипломната работа са изпълнени изцяло.

Използвана литература

[1] Simpson, Philip. FPGA design. Springer, 2010.

[2] P. P. Chu. FPGA Prototyping by VHDL Examples. John Wiley & Sons, 2018.

[3] S. Kilts. Advanced FPGA Design: Architecture, Implementation, and Optimization. Wiley-IEEE Press, 2007.

[4] Pele, Zoltan, et al. "One application of FPGA integrated circuits CMOS camera and LCD display controller design." 6th International Conference on Telecommunications in Modern Satellite, Cable and Broadcasting Service, 2003. TELSIKS 2003.. Vol. 1. IEEE, 2003.

[5] Aftab, Khan Huma, and Monauwer Alam. Design of VGA Controller using VHDL for LCD Display using FPGA. Diss. PhD thesis, University, India, 2014.

[6] Jiang, Xiu. "The design of LCD controller and its improved application based on FPGA." China-Ireland International Conference on Information and Communications Technologies (CIICT 2007). Stevenage UK: IET, 2007.

[7] Chen, Hao, Yu Chen, and Douglas H. Summerville. "A survey on the application of FPGAs for network infrastructure security." IEEE Communications Surveys & Tutorials 13.4 (2010): 541-561.

[8] Nasri Sulaiman, Zeyad Assi Obaid, Mohammad Hamiruce Marhaban, and Mohd Nizar Hamidon. "Design and implementation of FPGA-based systems-a review." Australian Journal of Basic and Applied Sciences 3.4 (2009): 3575-3596.

[9] Howard Jones. fpga-vt - FPGA VT-compatible terminal implementation, Available at: https://github.com/howardjones/fpga-vt

[10] DIY VT100 - A Miniature Hardware Terminal, Available at: https://hackaday.io/project/13273-diy-vt100-a-miniature-hardware-

terminal

[11] VT2040, Available at: https://hackaday.io/project/189699-vt2040

[12] FPGA serial terminal. Available at: https://hackaday.io/project/7165-fpga-serial-terminal

[13] 6802 serial terminal, Available at: https://hackaday.io/project/176175-6802-serial-terminal

[14] IEEE Standards Association. IEEE Std 1076 - VHDL Language Reference Manual, 2019.

[15] Xilinx Inc. Vivado Design Suite User Guide: Logic Simulation (UG900).

[16] Xilinx Inc. Vivado Design Suite User Guide: Programming and Debugging (UG908).

[17] Xilinx Inc. 7 Series FPGAs Data Sheet: Overview (DS180).

[18] Digi-Key Electronics. ps2_keyboard_to_ascii - PS/2 Keyboard to ASCII Converter (VHDL module).

[19] Jakub Cabal. UART controller (VHDL implementation). Open-source module.

[20] MikroTik. RouterOS Serial Console Documentation. Available at: https://help.mikrotik.com/docs/spaces/ROS/pages/328139/Serial+Console

[21] Geoff Graham. ASCII Video Terminal, Available at: https://geoffg.net/terminal.html

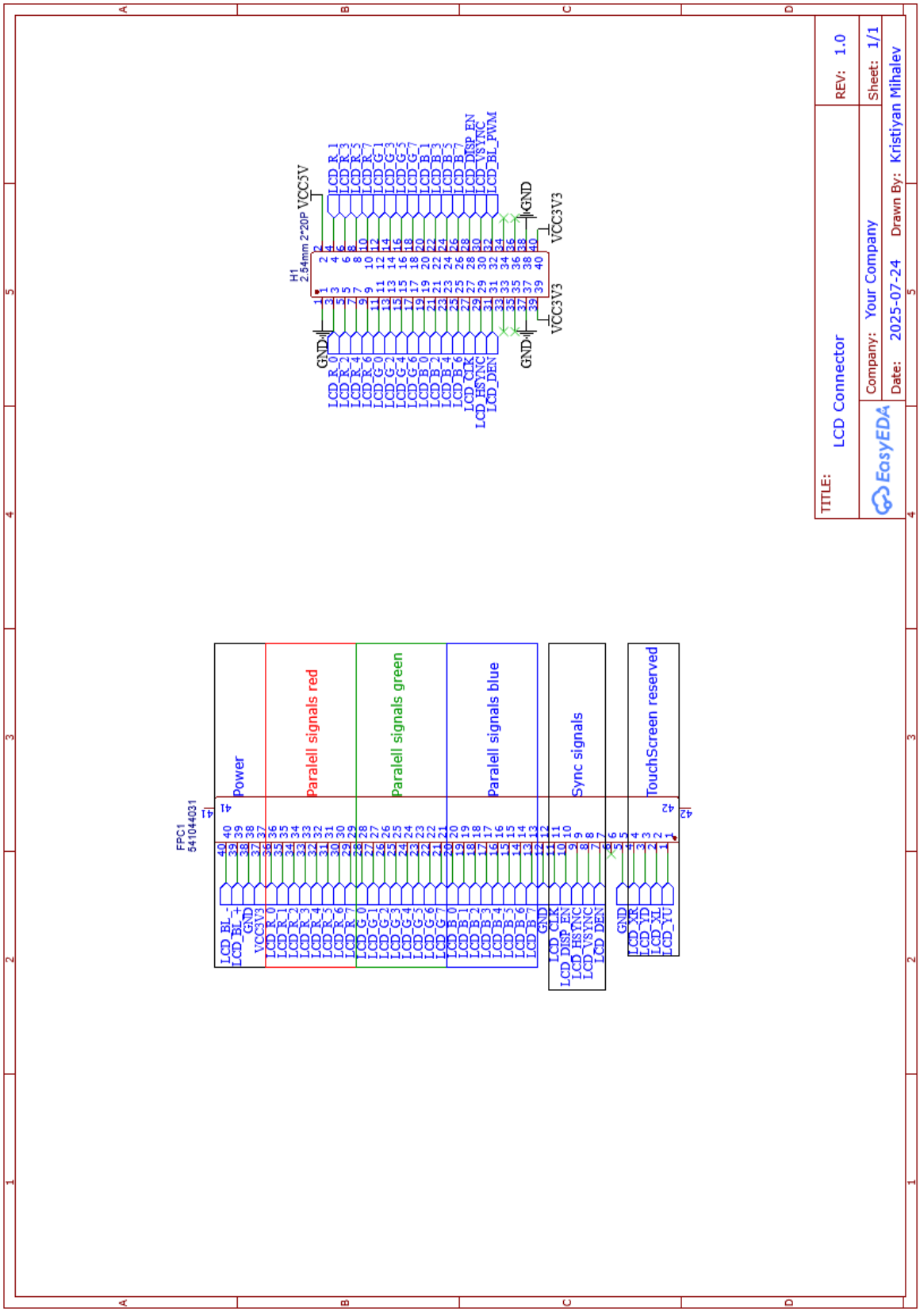
Спецификация

ID	Name	Designator	Footprint	Quantity	Manufacturer Part	Manufacturer
1	4.7uF	C1,C6,C7	C0805	3	CL21B475KOFNNNE	SAMSUNG(三星)
2	100nF	C2,C3,C5,C8,C13	C0402	5	CGA0402X7R104K160GT	HRE(芯声)
3	47nF	C9	C0603	1	C0603C473K5RAC7867	KEMET(基美)
4	33nF	C10,C11,C12	C0603	3	C0603C333K4RACTU	KEMET(基美)
5	HEADER 1x4	CN1	HEADER 1X4	1		
6	SS14G	D1	SMA_L4.3-W2.6-LS5.2-RD	1	SS14G	SHIKUES(时科)
7	541044031	FPC1	FPC-SMD_40P-P0.50_LSSJ-H2.0-40P	1	5,41E+08	MOLEX
8	2.54mm 2*20P	H1	HDR-TH_40P-P2.54-V-F-R2-C20-S2.54-2	1	2.54-2*20P母长PC104	BOOMELE(博穆精密)
9	FH-00124	H2,H3,H4,H5	CONN-TH_FH-00124	4	FH-00124	Liansheng(连盛)
10	HOLEM3	H6,H7,H8,H9	HOLEM3	4		
11	Connector_USB_A_Stacked	J1	A_HDJ_LIBRARY:USB_A_CUI_UJ2-ADH-TH_HORIZONTAL_STACKED	1		
12	4.7uH	L1	L1210	1	LQH32CN4R7M23L	muRata(村田)
13	12.4Ω	R1	R1210	1	AC1210FR-0712R4L	YAGEO(国巨)
14	0Ω	R4,R5,R6,R7	R0402	4	RC0402FR-070RL	YAGEO(国巨)
15	2301843-1	U1	DB9-TH_DB9FEMALE_C9900003585	1	2301843-1	TE Connectivity(泰科电子)

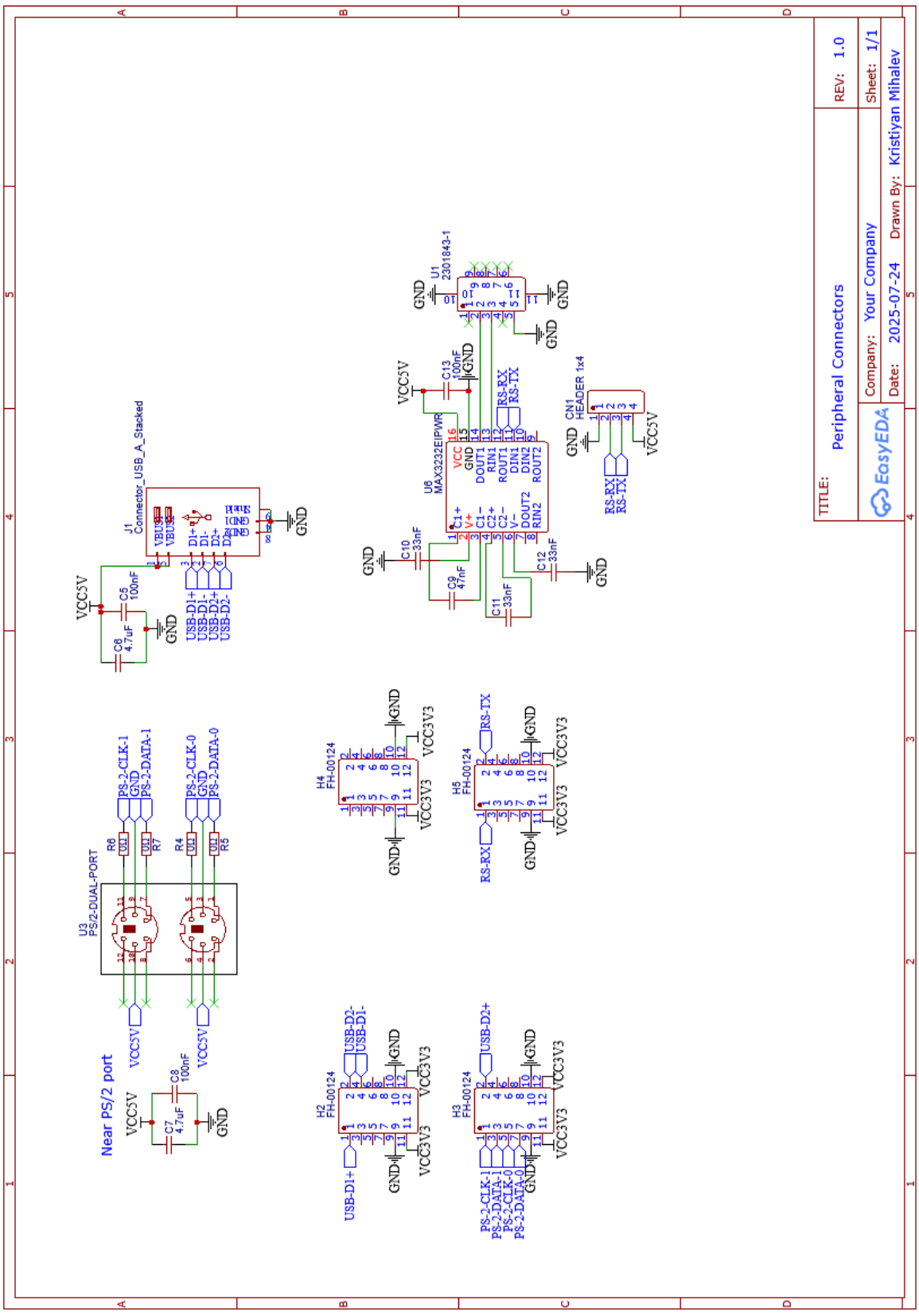
16	PS/2-DUAL-PORT	U3	PS/2-DOUBLE-PORT	1		
17	TPS61042DRB	U4	QFN-8_L3.0-W3.0-P0.65-TL-EP	1	TPS61042DRBR	TI(德州仪器)
18	1uF	U5	C1210	1	CKG32KX7R2A105M335AH	TDK
19	MAX3232EIPWR	U6	TSSOP-16_L5.0-W4.4-P0.65-LS6.4-BL	1	MAX3232EIPWR	TI(德州仪器)

Приложения

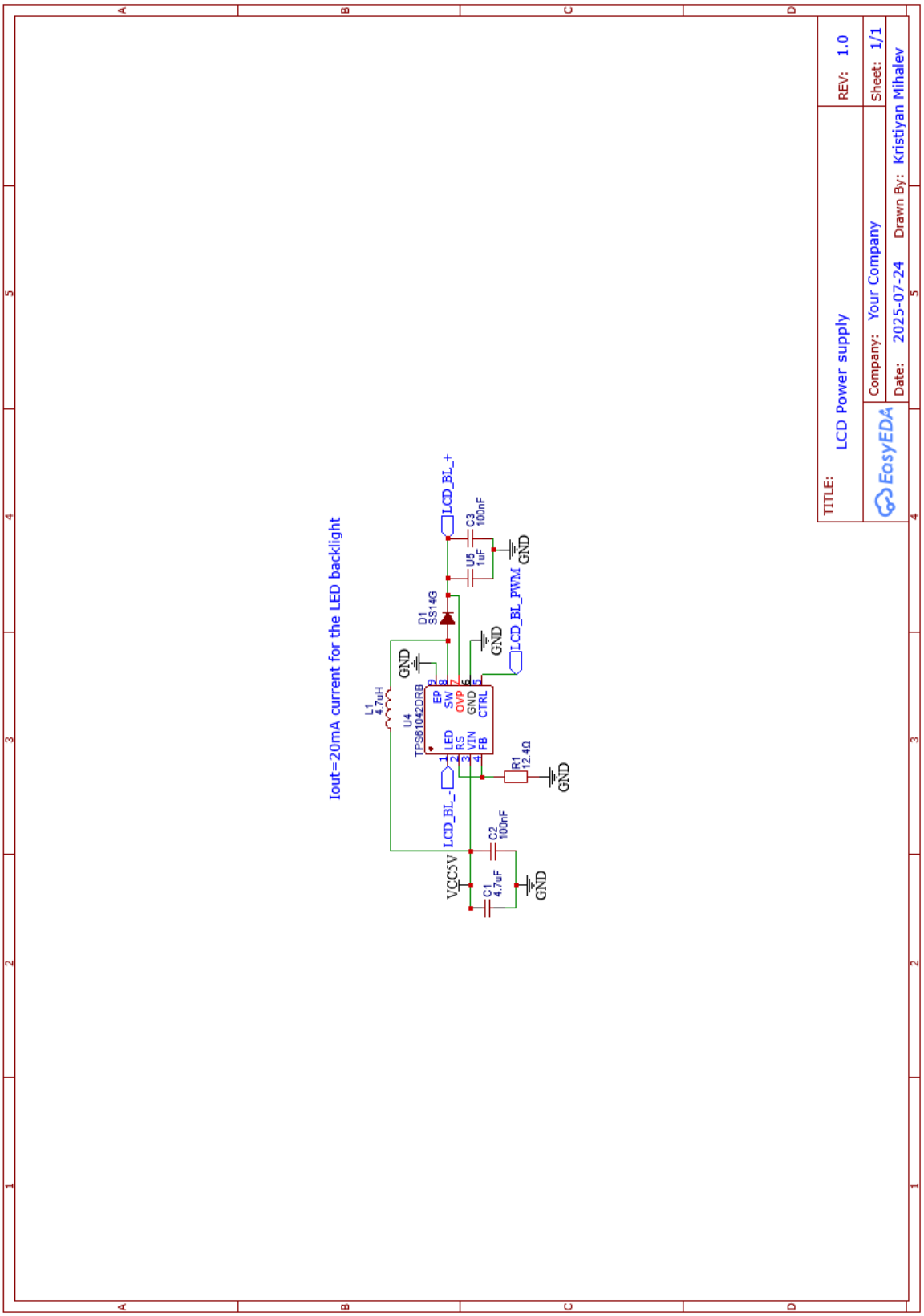
Приложение 1 - ел. схема на LCD конектора



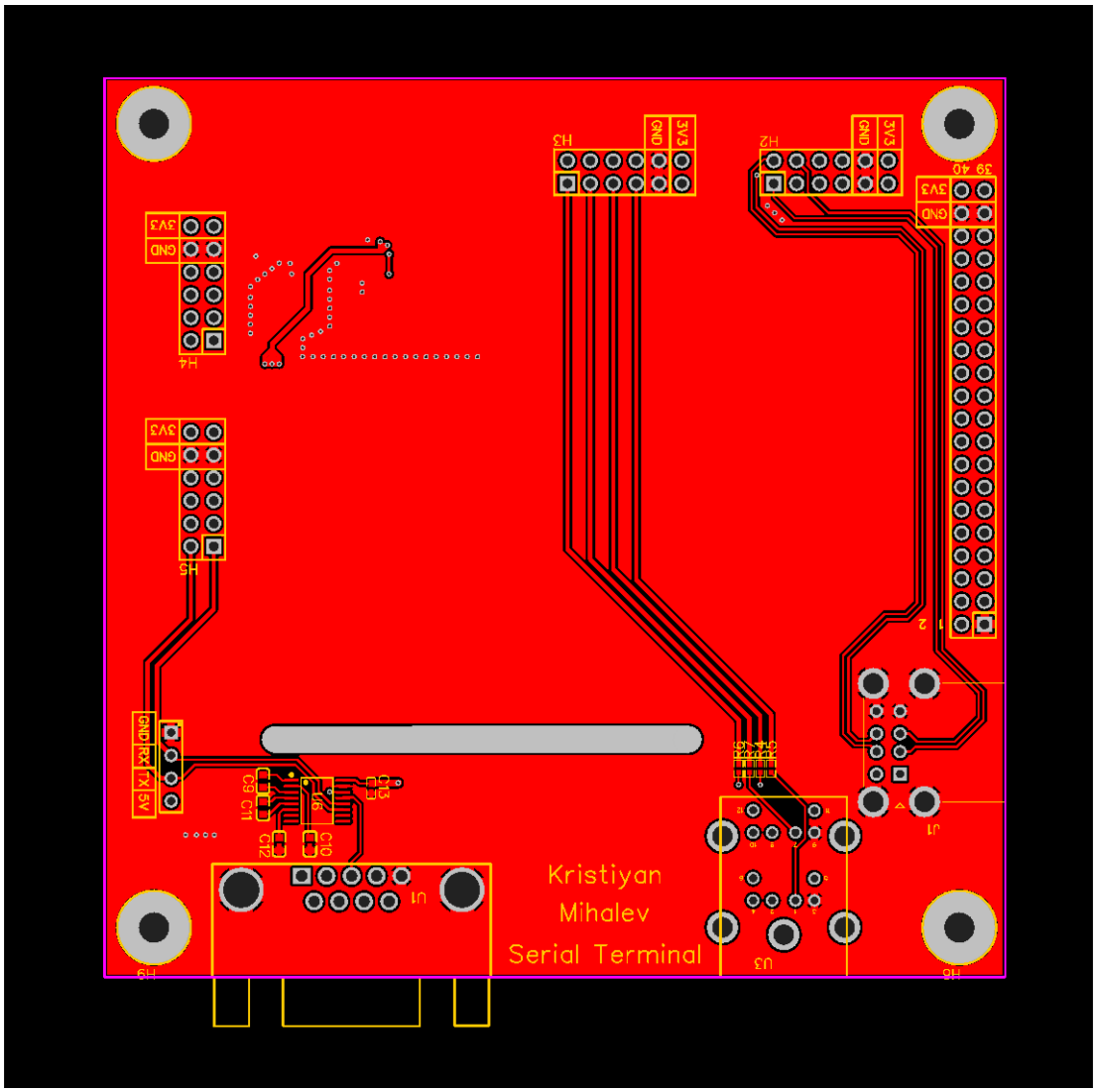
Приложение 2 - ел. схема на конекторите и RS-232 преобразувателя



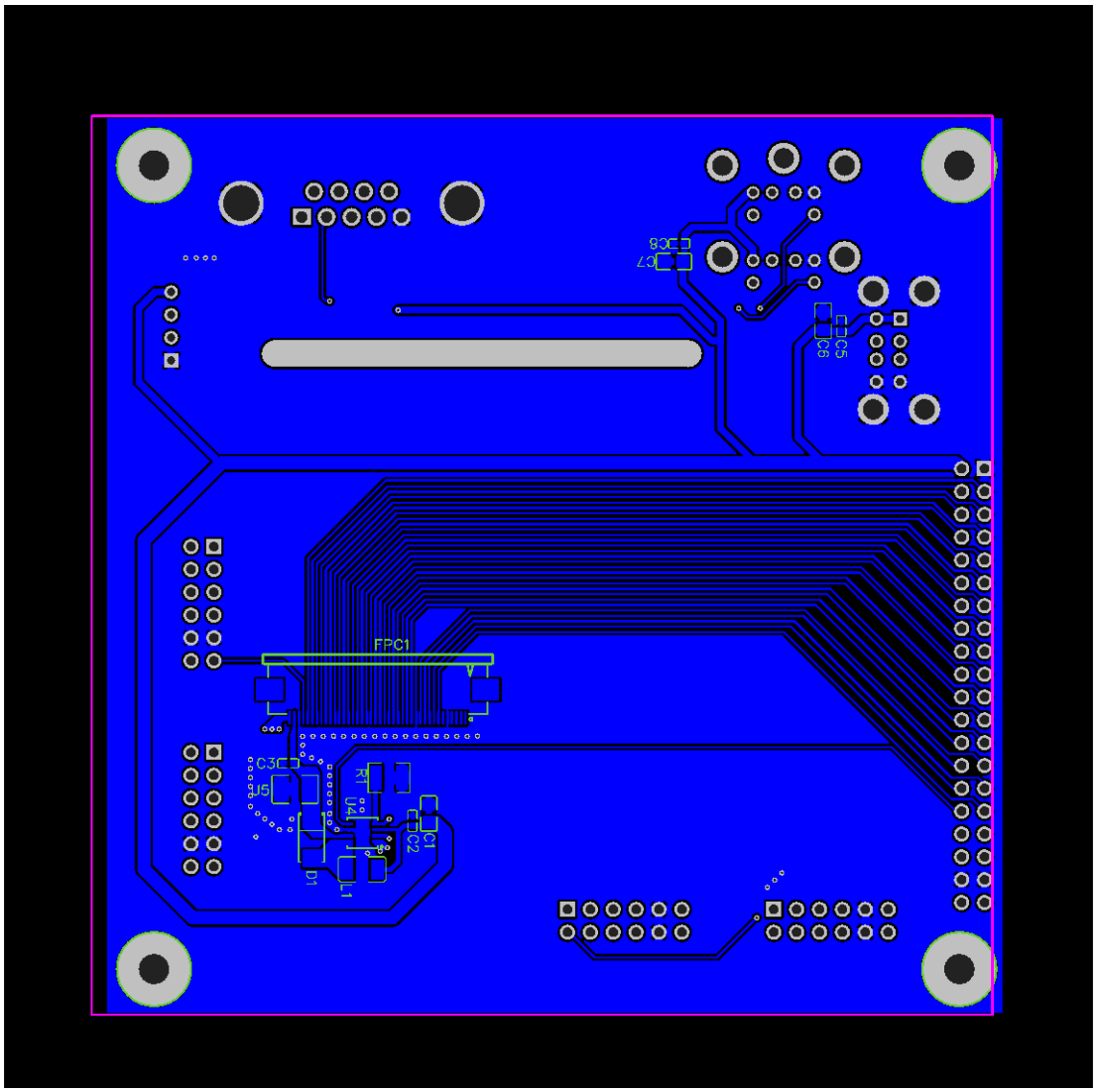
Приложение 3 - ел.схема на захранването за подсветката на LCD екрана



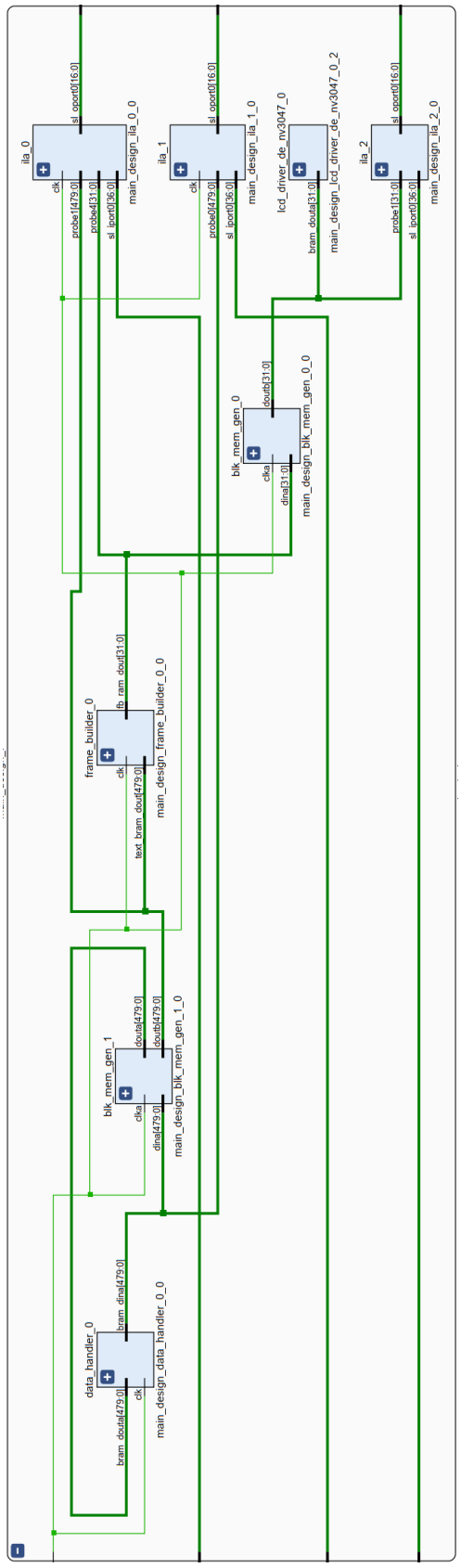
Приложение 4 - Горен слой на печатната платка



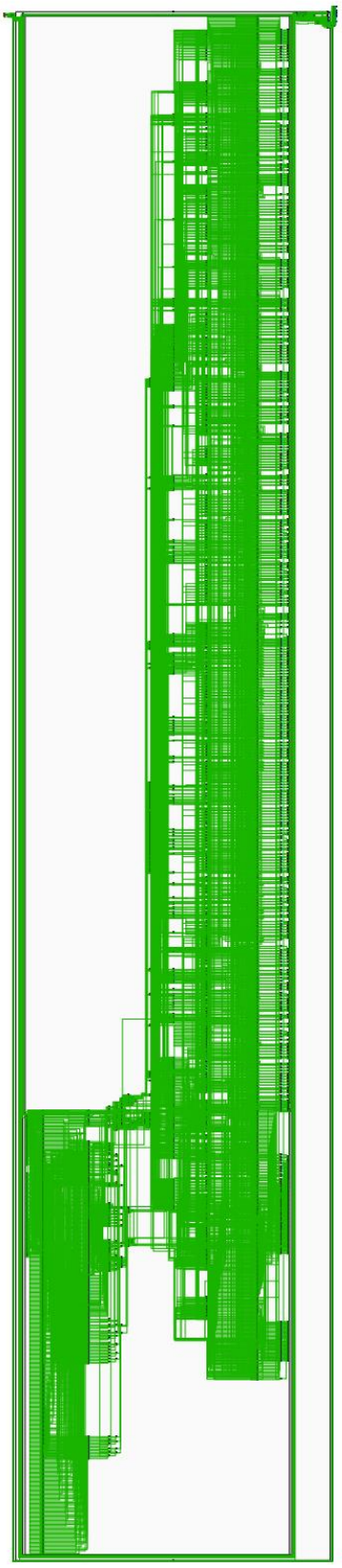
Приложение 5 - Долен слой на печатната платка



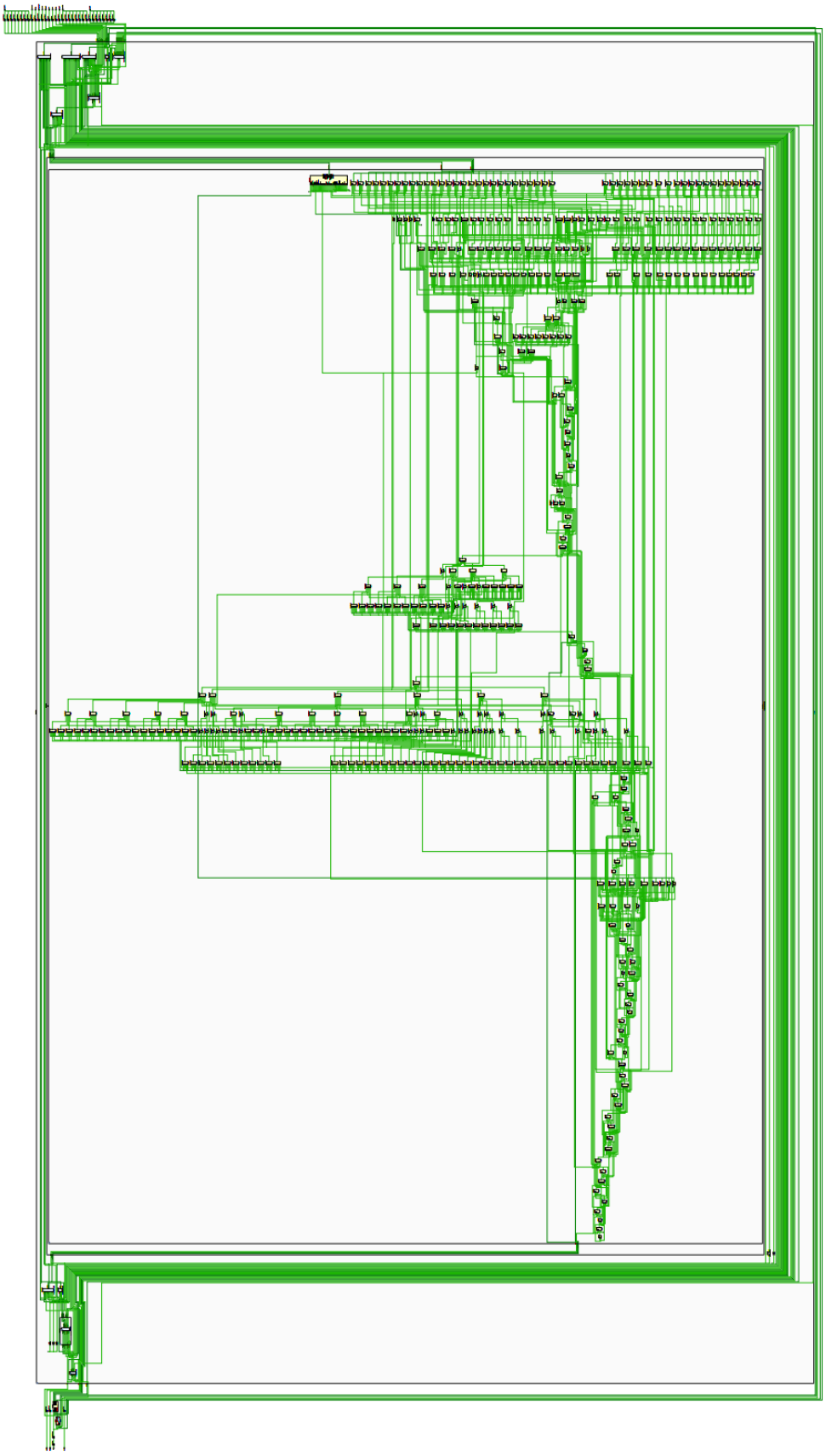
Приложение 6 - - диаграма на top-level модула



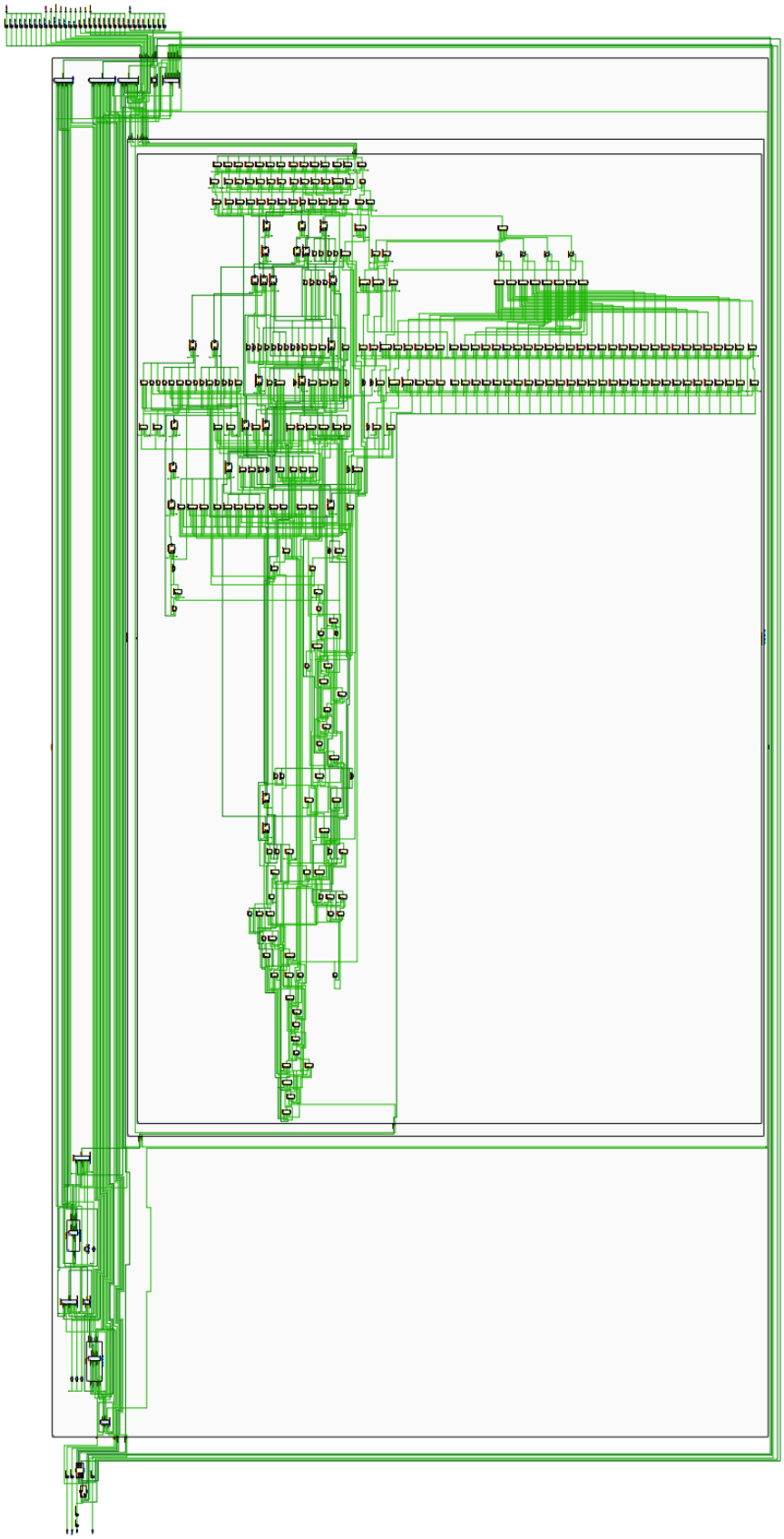
Приложение 7 - RTL дизайн на модул data_handler.vhd



Приложение 8 - RTL дизайн на модул frame_builder.vhd



Приложение 9 - RTL дизайн на модула lcd_driver.vhd



Приложение 10 - изображения на дъщерната платка



